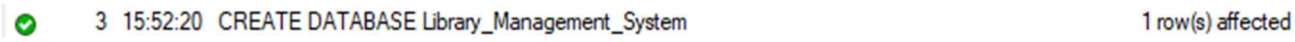
LIBRARY MANAGEMENT SYSTEM

SQL Project Report

CREATE DATABASE

```
CREATE DATABASE Library_Management_System;
```



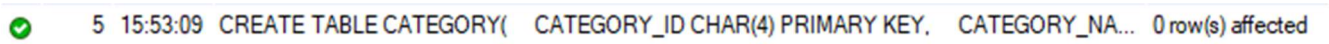
USE THE CREATED DATABASE

```
USE Library_Management_System;
```



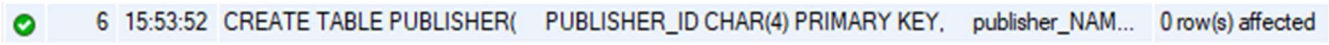
CREATE CATEGORY TABLE

```
CREATE TABLE CATEGORY(
    CATEGORY_ID CHAR(4) PRIMARY KEY,
    CATEGORY_NAME VARCHAR(100) NOT NULL
);
```



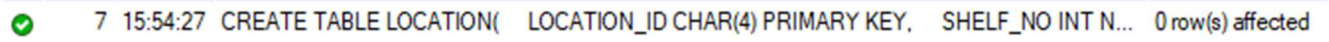
CREATE PUBLISHER TABLE

```
CREATE TABLE PUBLISHER(
    PUBLISHER_ID CHAR(4) PRIMARY KEY,
    publisher_NAME VARCHAR(100) NOT NULL,
    PUBLICATION_LANGUAGE VARCHAR(50),
    PUBLICATION_TYPE VARCHAR(50)
);
```



CREATE LOCATION TABLE

```
CREATE TABLE LOCATION(
    LOCATION_ID CHAR(4) PRIMARY KEY,
    SHELF_NO INT NOT NULL,
    SHELF_NAME VARCHAR(50),
    FLOOR_NO INT
);
```



CREATE AUTHOR TABLE

```
CREATE TABLE AUTHOR(
    AUTHOR_ID CHAR(4) PRIMARY KEY,
    FIRST_NAME VARCHAR(50) NOT NULL,
    LAST_NAME VARCHAR(50) NOT NULL
);
```

```
7  15:54:27  CREATE TABLE LOCATION(    LOCATION_ID CHAR(4) PRIMARY KEY,    SHELF_NO INT N...    0 row(s) affected
```

CREATE BOOK TABLE

```
CREATE TABLE BOOK(
    BOOK_ID CHAR(4) PRIMARY KEY,
    ISBN_CODE VARCHAR(20) UNIQUE NOT NULL,
    BOOK_TITLE VARCHAR(150) NOT NULL,

    CATEGORY_ID CHAR(4),
    PUBLISHER_ID CHAR(4),

    PUBLICATION_YEAR YEAR,
    BOOK_EDITION VARCHAR(20),

    COPIES_TOTAL INT NOT NULL,
    COPIES_AVAILABLE INT NOT NULL,

    LOCATION_ID CHAR(4),

    FOREIGN KEY (CATEGORY_ID)
      REFERENCES CATEGORY(CATEGORY_ID),

    FOREIGN KEY (PUBLISHER_ID)
      REFERENCES PUBLISHER(PUBLISHER_ID),

    FOREIGN KEY (LOCATION_ID)
      REFERENCES LOCATION(LOCATION_ID)
);
```

```
9  15:55:47  CREATE TABLE BOOK(    BOOK_ID CHAR(4) PRIMARY KEY,    ISBN_CODE VARCHAR(20) ...    0 row(s) affected
```

CREATE BOOK_AUTHOR TABLE

```
CREATE TABLE BOOK_AUTHOR(
    BOOK_ID CHAR(4),
    AUTHOR_ID CHAR(4),

    PRIMARY KEY (BOOK_ID, AUTHOR_ID),
```

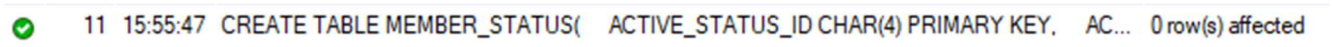
```
    FOREIGN KEY (BOOK_ID)
      REFERENCES BOOK(BOOK_ID),

    FOREIGN KEY (AUTHOR_ID)
      REFERENCES AUTHOR(AUTHOR_ID)
);
```



CREATE MEMBER_STATUS TABLE

```
CREATE TABLE MEMBER_STATUS(
    ACTIVE_STATUS_ID CHAR(4) PRIMARY KEY,
    ACCOUNT_TYPE ENUM('Student','Faculty','General'),
    ACCOUNT_STATUS ENUM('Active','Unactive','Removed','Expired'),
    MEMBERSHIP_START_DATE DATE,
    MEMBERSHIP_END_DATE DATE
);
```



CREATE MEMBERS TABLE

```
CREATE TABLE MEMBERS(
    MEMBER_ID CHAR(4) PRIMARY KEY,
    FIRST_NAME VARCHAR(50) NOT NULL,
    LAST_NAME VARCHAR(50) NOT NULL,
    CITY VARCHAR(50),

    PHONE_NO VARCHAR(20) UNIQUE,
    EMAIL_ID VARCHAR(100) UNIQUE,

    DATE_OF_BIRTH DATE,

    ACTIVE_STATUS_ID CHAR(4),

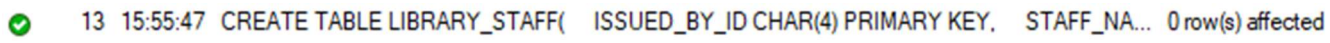
    FOREIGN KEY (ACTIVE_STATUS_ID)
      REFERENCES MEMBER_STATUS(ACTIVE_STATUS_ID)
);
```



CREATE LIBRARY_STAFF TABLE

```
CREATE TABLE LIBRARY_STAFF(
```

```
    ISSUED_BY_ID CHAR(4) PRIMARY KEY,
    STAFF_NAME VARCHAR(100) NOT NULL,
    STAFF_DESIGNATION VARCHAR(50)
);
```



CREATE BOOK_ISSUE TABLE

```
CREATE TABLE BOOK_ISSUE(
    ISSUE_ID CHAR(4) PRIMARY KEY,

    BOOK_ID CHAR(4),
    MEMBER_ID CHAR(4),

    ISSUE_DATE DATE NOT NULL,
    RETURN_DATE DATE,

    ISSUE_STATUS VARCHAR(20),

    ISSUED_BY_ID CHAR(4),

    FOREIGN KEY (BOOK_ID)
     REFERENCES BOOK(BOOK_ID),

    FOREIGN KEY (MEMBER_ID)
     REFERENCES MEMBERS(MEMBER_ID),

    FOREIGN KEY (ISSUED_BY_ID)
     REFERENCES LIBRARY_STAFF(ISSUED_BY_ID)
);
```



CREATE FINE_DUE TABLE

```
CREATE TABLE FINE_DUE(
    FINE_ID CHAR(4) PRIMARY KEY,

    MEMBER_ID CHAR(4),
    ISSUE_ID CHAR(4),

    FINE_DATE DATE,
```

```
    FINE_TOTAL DECIMAL(10,2) CHECK(FINE_TOTAL > 0),

    FOREIGN KEY (MEMBER_ID)
      REFERENCES MEMBERS(MEMBER_ID),

    FOREIGN KEY (ISSUE_ID)
      REFERENCES BOOK_ISSUE(ISSUE_ID)
);
```

```
15  15:55:48  CREATE TABLE FINE_DUE(    FINE_ID CHAR(4) PRIMARY KEY,    MEMBER_ID CHAR(4),  ...  0 row(s) affected
```

CREATE FINE_PAYMENT TABLE

```
CREATE TABLE FINE_PAYMENT(
    FINE_PAYMENT_ID CHAR(4) PRIMARY KEY,

    MEMBER_ID CHAR(4),

    PAYMENT_DATE DATE,
    PAYMENT_AMOUNT DECIMAL(10,2) CHECK(PAYMENT_AMOUNT > 0),

    FOREIGN KEY (MEMBER_ID)
      REFERENCES MEMBERS(MEMBER_ID)
);
```

```
16  15:55:48  CREATE TABLE FINE_PAYMENT(    FINE_PAYMENT_ID CHAR(4) PRIMARY KEY,    MEMB...  0 row(s) affected
```

```
INSERT INTO CATEGORY
(CATEGORY_ID, CATEGORY_NAME)
VALUES
('C001', 'Fiction'),
('C002', 'Science'),
('C003', 'History'),
('C004', 'Technology'),
('C005', 'Biography'),
('C006', 'Business'),
('C007', 'Literature'),
('C008', 'Mathematics');
```

```
17  16:00:40  INSERT INTO CATEGORY (CATEGORY_ID, CATEGORY_NAME) VALUES ('C001', 'Fiction'), ('...  8 row(s) affected Records: 8  Duplicates: 0  Warnings: 0
```

```
INSERT INTO PUBLISHER
(PUBLISHER_ID, PUBLISHER_NAME, PUBLICATION_LANGUAGE, PUBLICATION_TYPE)
```

```
VALUES
('P001', 'Penguin Random House', 'English', 'International'),
('P002', 'Oxford University Press', 'English', 'Academic'),
('P003', 'HarperCollins', 'English', 'International'),
('P004', 'Pearson Education', 'English', 'Academic'),
('P005', 'Sahitya Akademi', 'Hindi', 'Literary'),
('P006', 'McGraw Hill', 'English', 'Academic'),
('P007', 'Rupa Publications', 'English', 'Indian'),
('P008', 'Tata McGraw Hill', 'English', 'Academic');
```

17 16:00:40 INSERT INTO CATEGORY (CATEGORY_ID, CATEGORY_NAME) VALUES ('C001', 'Fiction'), ('... 8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0

```
INSERT INTO LOCATION
(LOCATION_ID, SHELF_NO, SHELF_NAME, FLOOR_NO)
VALUES
('L001', 101, 'Fiction Section', 1),
('L002', 102, 'Science Section', 1),
('L003', 201, 'History Section', 2),
('L004', 202, 'Technology Section', 2),
('L005', 301, 'Biography Section', 3),
('L006', 302, 'Business Section', 3),
('L007', 401, 'Literature Section', 4),
('L008', 402, 'Mathematics Section', 4);
```

19 16:00:40 INSERT INTO LOCATION (LOCATION_ID, SHELF_NO, SHELF_NAME, FLOOR_NO) VALUES... 8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0

```
INSERT INTO AUTHOR
(AUTHOR_ID, FIRST_NAME, LAST_NAME)
VALUES
('A001', 'George', 'Orwell'),
('A002', 'J.K.', 'Rowling'),
('A003', 'Stephen', 'Hawking'),
('A004', 'Yuval Noah', 'Harari'),
('A005', 'R.K.', 'Narayan'),
('A006', 'Robert', 'Kiyosaki'),
('A007', 'A.P.J. Abdul', 'Kalam'),
('A008', 'Chetan', 'Bhagat'),
('A009', 'Isaac', 'Asimov'),
('A010', 'Charles', 'Dickens');
```

20 16:00:40 INSERT INTO AUTHOR (AUTHOR_ID, FIRST_NAME, LAST_NAME) VALUES ('A001', 'Georg... 10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

```
-- Change YEAR to SMALLINT
ALTER TABLE BOOK
MODIFY COLUMN PUBLICATION_YEAR SMALLINT;
```

```
23  16:05:50  ALTER TABLE BOOK MODIFY COLUMN PUBLICATION_YEAR SMALLINT    0 row(s) affected Records: 0  Duplicates: 0  Warnings: 0
```

```
INSERT INTO BOOK
(
    BOOK_ID,
    ISBN_CODE,
    BOOK_TITLE,
    CATEGORY_ID,
    PUBLISHER_ID,
    PUBLICATION_YEAR,
    BOOK_EDITION,
    COPIES_TOTAL,
    COPIES_AVAILABLE,
    LOCATION_ID
)
VALUES
('B001', '9780451524935', '1984',
 'C001', 'P001', 1949, '1st Edition', 10, 7, 'L001'),

('B002', '9780747532699', 'Harry Potter and the Philosophers Stone',
 'C001', 'P003', 1997, '1st Edition', 12, 10, 'L001'),

('B003', '9780553380163', 'A Brief History of Time',
 'C002', 'P001', 1988, '2nd Edition', 8, 6, 'L002'),

('B004', '9780062316097', 'Sapiens',
 'C003', 'P003', 2015, '1st Edition', 10, 8, 'L003'),

('B005', '9788184001524', 'Wings of Fire',
 'C005', 'P007', 1999, '1st Edition', 15, 12, 'L005'),

('B006', '9781612680194', 'Rich Dad Poor Dad',
 'C006', 'P003', 1997, '1st Edition', 10, 8, 'L006'),

('B007', '9788184000713', 'The Guide',
 'C007', 'P007', 1958, '1st Edition', 7, 5, 'L007'),

('B008', '9780262033848', 'Introduction to Algorithms',
 'C004', 'P004', 2009, '3rd Edition', 8, 6, 'L004'),

('B009', '9780132350884', 'Clean Code',
 'C004', 'P004', 2008, '1st Edition', 10, 8, 'L004'),

('B010', '9780131103627', 'The C Programming Language',
 'C004', 'P004', 1988, '2nd Edition', 8, 6, 'L004'),

('B011', '9780199535569', 'Great Expectations',
```

```
 'C007', 'P002', 1861, '2nd Edition', 6, 4, 'L007'),

('B012', '9780061120084', 'The Alchemist',
 'C001', 'P003', 1988, '1st Edition', 10, 8, 'L001');
```

```
24  16:08:14  INSERT INTO BOOK (  BOOK_ID,  ISBN_CODE,  BOOK_TITLE,  CATEGORY_ID,  ...  12 row(s) affected Records: 12  Duplicates: 0  Warnings: 0
```

```
 INSERT INTO BOOK_AUTHOR
(BOOK_ID, AUTHOR_ID)
VALUES
('B001', 'A001'),
('B002', 'A002'),
('B003', 'A003'),
('B004', 'A004'),
('B005', 'A007'),
('B006', 'A006'),
('B007', 'A005'),
('B008', 'A009'),
('B009', 'A009'),
('B010', 'A010'),
('B011', 'A010'),
('B012', 'A008');
```

```
25  16:08:14  INSERT INTO BOOK_AUTHOR (BOOK_ID, AUTHOR_ID) VALUES ('B001', 'A001'), ('B002'...  12 row(s) affected Records: 12  Duplicates: 0  Warnings: 0
```

```
INSERT INTO MEMBER_STATUS
(
    ACTIVE_STATUS_ID,
    ACCOUNT_TYPE,
    ACCOUNT_STATUS,
    MEMBERSHIP_START_DATE,
    MEMBERSHIP_END_DATE
)
VALUES
('S001', 'Student', 'Active', '2026-01-01', '2026-12-31'),

('S002', 'Student', 'Active', '2026-02-15', '2027-02-14'),

('S003', 'Faculty', 'Active', '2026-01-01', '2027-12-31'),

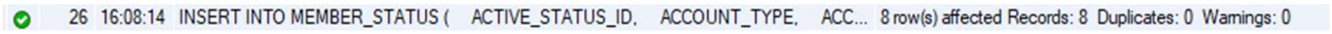
('S004', 'General', 'Active', '2026-03-01', '2027-02-28'),

('S005', 'Student', 'Expired', '2025-01-01', '2025-12-31'),

('S006', 'General', 'Unactive', '2025-06-01', '2026-05-31'),
```

```
('S007', 'Faculty', 'Active', '2026-04-01', '2027-03-31'),

('S008', 'Student', 'Active', '2026-05-01', '2027-04-30');
```




```
INSERT INTO MEMBERS
(
    MEMBER_ID,
    FIRST_NAME,
    LAST_NAME,
    CITY,
    PHONE_NO,
    EMAIL_ID,
    DATE_OF_BIRTH,
    ACTIVE_STATUS_ID
)
VALUES
('M001', 'Rahul', 'Sharma', 'Mumbai',
 '9876543210', 'rahul.sharma@gmail.com', '2003-05-15', 'S001'),

('M002', 'Priya', 'Patel', 'Mumbai',
 '9876543211', 'priya.patel@gmail.com', '2002-08-21', 'S002'),

('M003', 'Amit', 'Joshi', 'Thane',
 '9876543212', 'amit.joshi@gmail.com', '1985-02-10', 'S003'),

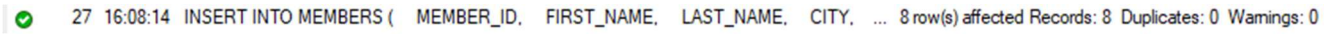
('M004', 'Sneha', 'Desai', 'Mumbai',
 '9876543213', 'sneha.desai@gmail.com', '1998-11-25', 'S004'),

('M005', 'Vikas', 'Mehta', 'Navi Mumbai',
 '9876543214', 'vikas.mehta@gmail.com', '2004-01-18', 'S005'),

('M006', 'Neha', 'Shah', 'Mumbai',
 '9876543215', 'neha.shah@gmail.com', '1995-07-12', 'S006'),

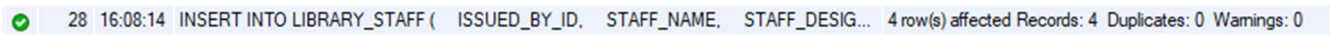
('M007', 'Rajesh', 'Kulkarni', 'Pune',
 '9876543216', 'rajesh.kulkarni@gmail.com', '1978-03-30', 'S007'),

('M008', 'Anjali', 'Gupta', 'Mumbai',
 '9876543217', 'anjali.gupta@gmail.com', '2001-09-05', 'S008');
```




```
 INSERT INTO LIBRARY_STAFF
```

```
(
    ISSUED_BY_ID,
    STAFF_NAME,
    STAFF_DESIGNATION
)
VALUES
('ST01', 'Rajesh Kumar', 'Chief Librarian'),
('ST02', 'Anita Sharma', 'Librarian'),
('ST03', 'Vijay Patil', 'Assistant Librarian'),
('ST04', 'Meena Joshi', 'Library Assistant');
```




```
INSERT INTO BOOK_ISSUE
(
    ISSUE_ID,
    BOOK_ID,
    MEMBER_ID,
    ISSUE_DATE,
    RETURN_DATE,
    ISSUE_STATUS,
    ISSUED_BY_ID
)
VALUES
('I001', 'B001', 'M001',
 '2026-09-01', '2026-09-10', 'Returned', 'ST01'),

('I002', 'B003', 'M002',
 '2026-09-03', '2026-09-12', 'Returned', 'ST02'),

('I003', 'B005', 'M003',
 '2026-09-05', '2026-09-15', 'Returned', 'ST01'),

('I004', 'B006', 'M004',
 '2026-09-07', NULL, 'Issued', 'ST03'),

('I005', 'B008', 'M001',
 '2026-09-08', NULL, 'Issued', 'ST02'),

('I006', 'B002', 'M007',
 '2026-09-09', '2026-09-18', 'Returned', 'ST04'),

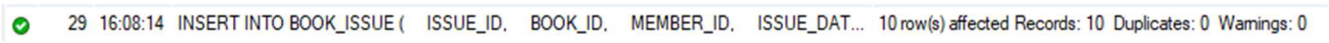
('I007', 'B009', 'M008',
 '2026-09-10', NULL, 'Issued', 'ST01'),

('I008', 'B004', 'M003',
```

```
 '2026-09-11', '2026-09-20', 'Returned', 'ST03'),

('I009', 'B010', 'M002',
 '2026-09-12', NULL, 'Issued', 'ST02'),

('I010', 'B012', 'M004',
 '2026-09-13', '2026-09-19', 'Returned', 'ST04');
```



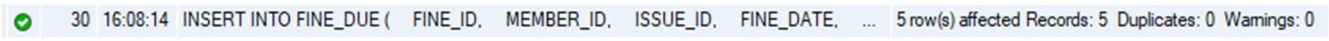

```
 INSERT INTO FINE_DUE
(
    FINE_ID,
    MEMBER_ID,
    ISSUE_ID,
    FINE_DATE,
    FINE_TOTAL
)
VALUES
('F001', 'M001', 'I001', '2026-09-11', 50.00),

('F002', 'M003', 'I003', '2026-09-16', 100.00),

('F003', 'M007', 'I006', '2026-09-19', 75.00),

('F004', 'M003', 'I008', '2026-09-21', 50.00),

('F005', 'M004', 'I010', '2026-09-20', 25.00);
```




```
INSERT INTO FINE_PAYMENT
(
    FINE_PAYMENT_ID,
    MEMBER_ID,
    PAYMENT_DATE,
    PAYMENT_AMOUNT
)
VALUES
('FP01', 'M001', '2026-09-12', 50.00),

('FP02', 'M003', '2026-09-17', 100.00),

('FP03', 'M007', '2026-09-20', 75.00),

('FP04', 'M004', '2026-09-21', 25.00);
```

31 16:08:15 INSERT INTO FINE_PAYMENT (FINE_PAYMENT_ID, MEMBER_ID, PAYMENT_... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0

```
SELECT * FROM CATEGORY;
```

CATEGORY_ID	CATEGORY_NAME
C001	Fiction
C002	Science
C003	History
C004	Technology
C005	Biography
C006	Business
C007	Literature
C008	Mathematics
NULL	NULL

```
SELECT * FROM PUBLISHER;
```

PUBLISHER_ID	publisher_NAME	PUBLICATION_LANGUAGE	PUBLICATION_TYPE
P001	Penguin Random House	English	International
P002	Oxford University Press	English	Academic
P003	HarperCollins	English	International
P004	Pearson Education	English	Academic
P005	Sahitya Akademi	Hindi	Literary
P006	McGraw Hill	English	Academic
P007	Rupa Publications	English	Indian
P008	Tata McGraw Hill	English	Academic
NULL	NULL	NULL	NULL

```
SELECT * FROM LOCATION;
```

LOCATION_ID	SHELF_NO	SHELF_NAME	FLOOR_NO
L001	101	Fiction Section	1
L002	102	Science Section	1
L003	201	History Section	2
L004	202	Technology Section	2
L005	301	Biography Section	3
L006	302	Business Section	3
L007	401	Literature Section	4
L008	402	Mathematics Section	4
NULL	NULL	NULL	NULL

```
SELECT * FROM AUTHOR;
```

AUTHOR_ID	FIRST_NAME	LAST_NAME
A001	George	Orwell
A002	J.K.	Rowling
A003	Stephen	Hawking
A004	Yuval Noah	Harari
A005	R.K.	Narayan
A006	Robert	Kiyosaki
A007	A.P.J. Abdul	Kalam
A008	Chetan	Bhagat
A009	Isaac	Asimov
A010	Charles	Dickens
NULL	NULL	NULL

```
SELECT * FROM BOOK;
```

BOOK_ID	ISBN_CODE	BOOK_TITLE	CATEGORY_ID	PUBLISHER_ID	PUBLICATION_YEAR	BOOK_EDITION	COPIES_TOTA
B001	9780451524935	1984	C001	P001	1949	1st Edition	10
B002	9780747532699	Harry Potter and the Philosophers Stone	C001	P003	1997	1st Edition	12
B003	9780553380163	A Brief History of Time	C002	P001	1988	2nd Edition	8
B004	9780062316097	Sapiens	C003	P003	2015	1st Edition	10
B005	9788184001524	Wings of Fire	C005	P007	1999	1st Edition	15
B006	9781612680194	Rich Dad Poor Dad	C006	P003	1997	1st Edition	10
B007	9788184000713	The Guide	C007	P007	1958	1st Edition	7
B008	9780262033848	Introduction to Algorithms	C004	P004	2009	3rd Edition	8
B009	9780132350884	Clean Code	C004	P004	2008	1st Edition	10
B010	9780131103627	The C Programming Language	C004	P004	1988	2nd Edition	8
B011	9780199535569	Great Expectations	C007	P002	1861	2nd Edition	6
B012	9780061120084	The Alchemist	C001	P003	1988	1st Edition	10
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
SELECT * FROM BOOK_AUTHOR;
```

BOOK_ID	AUTHOR_ID
B001	A001
B002	A002
B003	A003
B004	A004
B007	A005
B006	A006
B005	A007
B012	A008
B008	A009
B009	A009
B010	A010
B011	A010
NULL	NULL

```
SELECT * FROM MEMBER_STATUS;
```

ACTIVE_STATUS_ID	ACCOUNT_TYPE	ACCOUNT_STATUS	MEMBERSHIP_START_DATE	MEMBERSHIP_END_DATE
S001	Student	Active	2026-01-01	2026-12-31
S002	Student	Active	2026-02-15	2027-02-14
S003	Faculty	Active	2026-01-01	2027-12-31
S004	General	Active	2026-03-01	2027-02-28
S005	Student	Expired	2025-01-01	2025-12-31
S006	General	Unactive	2025-06-01	2026-05-31
S007	Faculty	Active	2026-04-01	2027-03-31
S008	Student	Active	2026-05-01	2027-04-30
NULL	NULL	NULL	NULL	NULL

SELECT * FROM MEMBERS;

MEMBER_ID	FIRST_NAME	LAST_NAME	CITY	PHONE_NO	EMAIL_ID	DATE_OF_BIRTH	ACTIVE_STATUS_ID
M001	Rahul	Sharma	Mumbai	9876543210	rahul.sharma@gmail.com	2003-05-15	S001
M002	Priya	Patel	Mumbai	9876543211	priya.patel@gmail.com	2002-08-21	S002
M003	Amit	Joshi	Thane	9876543212	amit.joshi@gmail.com	1985-02-10	S003
M004	Sneha	Desai	Mumbai	9876543213	sneha.desai@gmail.com	1998-11-25	S004
M005	Vikas	Mehta	Navi Mumbai	9876543214	vikas.mehta@gmail.com	2004-01-18	S005
M006	Neha	Shah	Mumbai	9876543215	neha.shah@gmail.com	1995-07-12	S006
M007	Rajesh	Kulkarni	Pune	9876543216	rajesh.kulkarni@gmail.com	1978-03-30	S007
M008	Anjali	Gupta	Mumbai	9876543217	anjali.gupta@gmail.com	2001-09-05	S008
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM LIBRARY_STAFF;

ISSUED_BY_ID	STAFF_NAME	STAFF_DESIGNATION
ST01	Rajesh Kumar	Chief Librarian
ST02	Anita Sharma	Librarian
ST03	Vijay Patil	Assistant Librarian
ST04	Meena Joshi	Library Assistant
NULL	NULL	NULL

SELECT * FROM BOOK_ISSUE;

ISSUE_ID	BOOK_ID	MEMBER_ID	ISSUE_DATE	RETURN_DATE	ISSUE_STATUS	ISSUED_BY_ID
I001	B001	M001	2026-09-01	2026-09-10	Returned	ST01
I002	B003	M002	2026-09-03	2026-09-12	Returned	ST02
I003	B005	M003	2026-09-05	2026-09-15	Returned	ST01
I004	B006	M004	2026-09-07	NULL	Issued	ST03
I005	B008	M001	2026-09-08	NULL	Issued	ST02
I006	B002	M007	2026-09-09	2026-09-18	Returned	ST04
I007	B009	M008	2026-09-10	NULL	Issued	ST01
I008	B004	M003	2026-09-11	2026-09-20	Returned	ST03
I009	B010	M002	2026-09-12	NULL	Issued	ST02
I010	B012	M004	2026-09-13	2026-09-19	Returned	ST04
NULL	NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM FINE_DUE;

	FINE_ID	MEMBER_ID	ISSUE_ID	FINE_DATE	FINE_TOTAL
▶	F001	M001	I001	2026-09-11	50.00
	F002	M003	I003	2026-09-16	100.00
	F003	M007	I006	2026-09-19	75.00
	F004	M003	I008	2026-09-21	50.00
	F005	M004	I010	2026-09-20	25.00
*	NULL	NULL	NULL	NULL	NULL

```
SELECT * FROM FINE_PAYMENT;
```

	FINE_PAYMENT_ID	MEMBER_ID	PAYMENT_DATE	PAYMENT_AMOUNT
▶	FP01	M001	2026-09-12	50.00
	FP02	M003	2026-09-17	100.00
	FP03	M007	2026-09-20	75.00
	FP04	M004	2026-09-21	25.00
*	NULL	NULL	NULL	NULL

DATA CLEANING

Dirty member data

```
CREATE TABLE MEMBERS_RAW (
    RAW_ID INT AUTO_INCREMENT PRIMARY KEY,
    MEMBER_CODE VARCHAR(20),
    FULL_NAME VARCHAR(100),
    CITY_STATE VARCHAR(100),
    PHONE_NO VARCHAR(30),
    EMAIL_ID VARCHAR(150),
    DATE_OF_BIRTH VARCHAR(30),
    ACCOUNT_TYPE VARCHAR(30),
    ACCOUNT_STATUS VARCHAR(30),
    MEMBERSHIP_START_DATE VARCHAR(30),
    MEMBERSHIP_END_DATE VARCHAR(30),
    TEMP_NOTES VARCHAR(100),
    IMPORT_BATCH VARCHAR(30)
);
```

✔	44	16:22:36	CREATE TABLE MEMBERS_RAW (	RAW_ID INT AUTO_INCREMENT PRIMARY KEY,...	0 row(s) affected

```
-- Insert the dirty records:
INSERT INTO MEMBERS_RAW
(
    MEMBER_CODE, FULL_NAME, CITY_STATE, PHONE_NO,
    EMAIL_ID, DATE_OF_BIRTH, ACCOUNT_TYPE, ACCOUNT_STATUS,
```

```
    MEMBERSHIP_START_DATE, MEMBERSHIP_END_DATE,
    TEMP_NOTES, IMPORT_BATCH
)
VALUES
('M001', '  Rahul Sharma  ', 'Mumbai, Maharashtra',
 '9876543210', 'rahul.sharma@gmail.com', '2003-05-15',
 'Student', 'Active', '2026-01-01', '2026-12-31',
 'Imported from Excel', 'BATCH001'),

('M002', 'Priya Patel', 'mumbai, maharashtra',
 '9876543211', 'priya.patel@gmail.com', '21/08/2002',
 'STUDENT', 'active', '15-02-2026', '14-02-2027',
 'Check date format', 'BATCH001'),

('M003', 'Amit Joshi', 'Thane, Maharashtra',
 '9876543212', 'amit.joshi@gmail.com', '1985-02-10',
 'Faculty', 'Active', '2026-01-01', '2027-12-31',
 'Verified', 'BATCH001'),

('M004', 'Sneha Desai', 'Mumbai, Maharashtra',
 '123', 'sneha.desai@gmail.com', '1998-11-25',
 'General', 'ACTIVE', '2026-03-01', '2027-02-28',
 'Invalid phone', 'BATCH002'),

('M005', 'Vikas Mehta', 'Navi Mumbai, Maharashtra',
 '9876543214', 'vikas.mehta@gmail.com', '2004-01-18',
 'Student', 'Expired', '2025-01-01', '2025-12-31',
 'Expired membership', 'BATCH002'),

('M006', 'Neha Shah', 'Mumbai, Maharashtra',
 '', 'neha.shah@gmail.com', '1995-07-12',
 'General', 'Unactive', '2025-06-01', '2026-05-31',
 'Missing phone', 'BATCH002'),

('M007', '  Rahul Sharma  ', 'Mumbai, Maharashtra',
 '9876543210', 'rahul.sharma@gmail.com', '2003-05-15',
 'Student', 'Active', '2026-01-01', '2026-12-31',
 'Duplicate record', 'BATCH003'),

('M008', 'Anjali Gupta', 'Mumbai, Maharashtra',
 NULL, 'anjali.gupta@gmail.com', '2001-09-05',
 'Student', 'Active', '2026-05-01', '2027-04-30',
 NULL, 'BATCH003'),

('M009', 'Karan Malhotra', 'Pune, Maharashtra',
 '9876543218', 'karan.malhotra@gmail.com', '31-13-2000',
 'student', 'active', '2026-02-30', '2027-02-28',
 'Invalid dates', 'BATCH003'),
```

```
('M010', 'Meera Iyer', '  Mumbai , Maharashtra  ',
 '98765ABC10', 'meera.iyer@gmail.com', '1999-06-20',
 'Faculty', 'Active', '2026-04-01', NULL,
 'Membership end missing', 'BATCH003'),

('M011', 'Arjun Rao', 'Thane, Maharashtra',
 '9876543219', 'arjun.rao.gmail.com', '2001-04-12',
 'Student', 'Active', '2026-01-01', '2026-12-31',
 'Invalid email', 'BATCH004'),

('M012', '   ', 'Mumbai, Maharashtra',
 '9876543220', NULL, '2000-10-10',
 'General', 'Active', '2026-01-01', '2026-12-31',
 'Missing name and email', 'BATCH004');
```



Dirty book data

```
 CREATE TABLE BOOK_RAW (
    RAW_ID INT AUTO_INCREMENT PRIMARY KEY,
    BOOK_CODE VARCHAR(20),
    ISBN_CODE VARCHAR(30),
    BOOK_TITLE VARCHAR(150),
    CATEGORY_NAME VARCHAR(50),
    PUBLISHER_NAME VARCHAR(100),
    AUTHOR_NAME VARCHAR(100),
    PUBLICATION_YEAR VARCHAR(30),
    BOOK_EDITION VARCHAR(40),
    COPIES_TOTAL VARCHAR(20),
    COPIES_AVAILABLE VARCHAR(20),
    SHELF_LOCATION VARCHAR(100),
    TEMP_COMMENT VARCHAR(100)
);
```




```
-- Insert the dirty records:
INSERT INTO BOOK_RAW
(
    BOOK_CODE, ISBN_CODE, BOOK_TITLE, CATEGORY_NAME,
    PUBLISHER_NAME, AUTHOR_NAME, PUBLICATION_YEAR,
    BOOK_EDITION, COPIES_TOTAL, COPIES_AVAILABLE,
    SHELF_LOCATION, TEMP_COMMENT
)
VALUES
```

```
('B001', '9780451524935', '  1984  ', 'Fiction',
 'Penguin Random House', 'George Orwell',
 '1949', '1st Edition', '10', '7',
 'L001 - Shelf 101', 'Imported record'),

('B002', '9780747532699',
 'Harry Potter and the Philosophers Stone',
 'fiction', 'HarperCollins', 'J.K. Rowling',
 '1997', '1st Edition', '12', '10',
 'L001 - Shelf 101', 'Category case mismatch'),

('B003', '9780553380163', 'A Brief History of Time',
 'Science', 'Penguin Random House', 'Stephen Hawking',
 '1988', '2nd Edition', 'Eight', '6',
 'L002 - Shelf 102', 'Invalid total copies'),

('B004', '9780062316097', 'Sapiens',
 'History', 'HarperCollins', 'Yuval Noah Harari',
 'Twenty Fifteen', '1st Edition', '10', '8',
 'L003 - Shelf 201', 'Invalid year'),

('B005', '9788184001524', 'Wings of Fire',
 'Biography', 'Rupa Publications', 'A.P.J. Abdul Kalam',
 '1999', '1st Edition', '15', '20',
 'L005 - Shelf 301', 'Available copies exceed total'),

('B006', '9781612680194', 'Rich Dad Poor Dad',
 'Business', 'HarperCollins', 'Robert Kiyosaki',
 '1997', '1st Edition', '-10', '8',
 'L006 - Shelf 302', 'Invalid negative total'),

('B007', '9788184000713', 'The Guide',
 'Literature', 'Rupa Publications', 'R.K. Narayan',
 '1958', '1st Edition', '7', '5',
 'L007 - Shelf 401', 'Valid record'),

('B008', '9780262033848', 'Introduction to Algorithms',
 'Technology', 'Pearson Education', 'Isaac Asimov',
 '2009', '3rd Edition', '8', '6',
 'L004 - Shelf 202', 'Check author assignment'),

('B009', '9780132350884', 'Clean Code',
 'Technology', 'Pearson Education', 'Isaac Asimov',
 '2008', '1st Edition', '10', '6',
 'L004 - Shelf 202', 'Valid sample'),

('B010', '9780131103627', 'The C Programming Language',
 'Technology', 'Pearson Education', 'Charles Dickens',
 '1988', '2nd Edition', '8', NULL,
```

```
 'L004 - Shelf 202', 'Missing available copies'),

('B011', '9780199535569', 'Great Expectations',
 'Literature', 'Oxford University Press', 'Charles Dickens',
 '1861', '2nd Edition', '6', '4',
 'L007 - Shelf 401', 'Valid sample'),

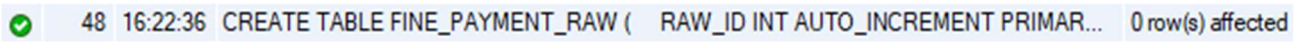
('B012', '9780451524935', '  1984  ', 'Fiction',
 'Penguin Random House', 'George Orwell',
 '1949', '1st Edition', '10', '7',
 'L001 - Shelf 101', 'Duplicate record'),

('B013', '9780061120084', 'The Alchemist',
 'Fiction', 'HarperCollins', 'Chetan Bhagat',
 '1988', '1st Edition', '10', '8',
 'L001 - Shelf 101', 'Check author assignment'),

('B014', '9780000000000', 'Unknown Book',
 'Sci-Fi', 'Unknown Publisher', 'Unknown Author',
 '20O5', 'First', 'Five', 'Two',
 'Shelf 9', NULL);
```


Dirty fine payment data

```
 CREATE TABLE FINE_PAYMENT_RAW (
    RAW_ID INT AUTO_INCREMENT PRIMARY KEY,
    PAYMENT_CODE VARCHAR(20),
    MEMBER_NAME VARCHAR(100),
    PAYMENT_DATE VARCHAR(30),
    PAYMENT_AMOUNT VARCHAR(30),
    PAYMENT_METHOD VARCHAR(30),
    TEMP_NOTES VARCHAR(100)
);
```




```
-- Insert the dirty records:
INSERT INTO FINE_PAYMENT_RAW
(
    PAYMENT_CODE, MEMBER_NAME, PAYMENT_DATE,
    PAYMENT_AMOUNT, PAYMENT_METHOD, TEMP_NOTES
)
VALUES
('FP01', 'Rahul Sharma', '2026-09-12',
 '50.00', 'Cash', 'Valid payment'),
```

```
('FP02', 'Amit Joshi', '17/09/2026',
 '100', 'UPI', 'Different date format'),

('FP03', 'Rajesh Kulkarni', '2026-09-20',
 '75.00', 'upi', 'Inconsistent payment method'),

('FP04', 'Sneha Desai', '2026-09-21',
 '25.00', 'Card', 'Valid payment'),

('FP05', 'Neha Shah', '2026-09-22',
 '-100', 'Cash', 'Negative payment'),

('FP06', 'Priya Patel', '2026-09-23',
 'One Hundred', 'UPI', 'Invalid amount'),

('FP07', 'Rahul Sharma', '2026-09-12',
 '50.00', 'Cash', 'Possible duplicate'),

('FP08', 'Karan Malhotra', '2026-09-24',
 NULL, 'UPI', 'Missing amount'),

('FP09', 'Meera Iyer', '2026-09-25',
 '200.00', NULL, 'Missing payment method'),

('FP10', 'Anjali Gupta', '2026-02-30',
 '150.00', 'Cash', 'Invalid date');
```

```
49  16:22:36  INSERT INTO FINE_PAYMENT_RAW (  PAYMENT_CODE, MEMBER_NAME, PAYMEN...  10 row(s) affected Records: 10  Duplicates: 0  Warnings: 0
```

```
-- Find duplicate member emails
SELECT
    LOWER(TRIM(EMAIL_ID)) AS EMAIL_ID,
    COUNT(*) AS DUPLICATE_COUNT
FROM MEMBERS_RAW
WHERE EMAIL_ID IS NOT NULL
  AND TRIM(EMAIL_ID) <> ''
GROUP BY LOWER(TRIM(EMAIL_ID))
HAVING COUNT(*) > 1;
```

	EMAIL_ID	DUPLICATE_COUNT
▶	rahul.sharma@gmail.com	2

```
-- Delete dirty data
DELETE M1
FROM MEMBERS_RAW M1
```

```
INNER JOIN MEMBERS_RAW M2
    ON LOWER(TRIM(M1.EMAIL_ID)) =
       LOWER(TRIM(M2.EMAIL_ID))
   AND M1.RAW_ID > M2.RAW_ID
WHERE M1.EMAIL_ID IS NOT NULL
  AND M2.EMAIL_ID IS NOT NULL
  AND TRIM(M1.EMAIL_ID) <> ''
  AND TRIM(M2.EMAIL_ID) <> '';
```

53 16:27:47 DELETE M1 FROM MEMBERS_RAW M1 INNER JOIN MEMBERS_RAW M2 ON LOW... 1 row(s) affected

```
-- Find duplicate books using ISBN
SELECT
    ISBN_CODE,
    COUNT(*) AS RECORD_COUNT
FROM BOOK_RAW
WHERE ISBN_CODE IS NOT NULL
  AND TRIM(ISBN_CODE) <> ''
GROUP BY ISBN_CODE
HAVING COUNT(*) > 1;
```

ISBN_CODE	RECORD_COUNT
9780451524935	2

```
SELECT *
FROM BOOK_RAW
WHERE ISBN_CODE = '9780451524935'
ORDER BY RAW_ID;
```

RAW_ID	BOOK_CODE	ISBN_CODE	BOOK_TITLE	CATEGORY_NAME	PUBLISHER_NAME	AUTHOR_NAME	PUBLICATION_YEAR	BOOK_EDITI
1	B001	9780451524935	1984	Fiction	Penguin Random House	George Orwell	1949	1st Edition
12	B012	9780451524935	1984	Fiction	Penguin Random House	George Orwell	1949	1st Edition
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
-- Delete dirty data
DELETE B1
FROM BOOK_RAW B1
INNER JOIN BOOK_RAW B2
    ON B1.ISBN_CODE = B2.ISBN_CODE
   AND B1.RAW_ID > B2.RAW_ID
WHERE B1.ISBN_CODE IS NOT NULL
  AND TRIM(B1.ISBN_CODE) <> '';
```

56 16:29:39 DELETE B1 FROM BOOK_RAW B1 INNER JOIN BOOK_RAW B2 ON B1.ISBN_CODE ... 1 row(s) affected

```
-- Find duplicate fine payments
SELECT
    PAYMENT_CODE,
    MEMBER_NAME,
    PAYMENT_DATE,
    PAYMENT_AMOUNT,
    PAYMENT_METHOD,
    COUNT(*) AS RECORD_COUNT
FROM FINE_PAYMENT_RAW
GROUP BY
    PAYMENT_CODE,
    MEMBER_NAME,
    PAYMENT_DATE,
    PAYMENT_AMOUNT,
    PAYMENT_METHOD
HAVING COUNT(*) > 1;
```

	PAYMENT_CODE	MEMBER_NAME	PAYMENT_DATE	PAYMENT_AMOUNT	PAYMENT_METHOD	RECORD_COUNT

```
-- Check data types
DESCRIBE MEMBERS_RAW;
```

	Field	Type	Null	Key	Default	Extra
▶	RAW_ID	int	NO	PRI	NULL	auto_increment
	MEMBER_CODE	varchar(20)	YES		NULL	
	FULL_NAME	varchar(100)	YES		NULL	
	CITY_STATE	varchar(100)	YES		NULL	
	PHONE_NO	varchar(30)	YES		NULL	
	EMAIL_ID	varchar(150)	YES		NULL	
	DATE_OF_BIRTH	varchar(30)	YES		NULL	
	ACCOUNT_TYPE	varchar(30)	YES		NULL	
	ACCOUNT_STATUS	varchar(30)	YES		NULL	
	MEMBERSHIP_START_DATE	varchar(30)	YES		NULL	
	MEMBERSHIP_END_DATE	varchar(30)	YES		NULL	
	TEMP_NOTES	varchar(100)	YES		NULL	
	IMPORT_BATCH	varchar(30)	YES		NULL	

```
DESCRIBE BOOK_RAW;
```

Field	Type	Null	Key	Default	Extra
RAW_ID	int	NO	PRI	NULL	auto_increment
BOOK_CODE	varchar(20)	YES		NULL	
ISBN_CODE	varchar(30)	YES		NULL	
BOOK_TITLE	varchar(150)	YES		NULL	
CATEGORY_NAME	varchar(50)	YES		NULL	
PUBLISHER_NAME	varchar(100)	YES		NULL	
AUTHOR_NAME	varchar(100)	YES		NULL	
PUBLICATION_YEAR	varchar(30)	YES		NULL	
BOOK_EDITION	varchar(40)	YES		NULL	
COPIES_TOTAL	varchar(20)	YES		NULL	
COPIES_AVAILABLE	varchar(20)	YES		NULL	
SHELF_LOCATION	varchar(100)	YES		NULL	
TEMP_COMMENT	varchar(100)	YES		NULL	

```
DESCRIBE FINE_PAYMENT_RAW;
```

Field	Type	Null	Key	Default	Extra
RAW_ID	int	NO	PRI	NULL	auto_increment
PAYMENT_CODE	varchar(20)	YES		NULL	
MEMBER_NAME	varchar(100)	YES		NULL	
PAYMENT_DATE	varchar(30)	YES		NULL	
PAYMENT_AMOUNT	varchar(30)	YES		NULL	
PAYMENT_METHOD	varchar(30)	YES		NULL	
TEMP_NOTES	varchar(100)	YES		NULL	

```
-- Check invalid member dates
SELECT RAW_ID, MEMBER_CODE, DATE_OF_BIRTH
FROM MEMBERS_RAW
WHERE DATE_OF_BIRTH IS NOT NULL
  AND DATE_OF_BIRTH <> ''
  AND STR_TO_DATE(DATE_OF_BIRTH, '%Y-%m-%d') IS NULL
  AND STR_TO_DATE(DATE_OF_BIRTH, '%d/%m/%Y') IS NULL
  AND STR_TO_DATE(DATE_OF_BIRTH, '%d-%m-%Y') IS NULL;
```

RAW_ID	MEMBER_CODE	DATE_OF_BIRTH
9	M009	31-13-2000
NULL	NULL	NULL

```
-- Check membership dates too:
SELECT
    RAW_ID,
    MEMBER_CODE,
    MEMBERSHIP_START_DATE,
    MEMBERSHIP_END_DATE
FROM MEMBERS_RAW
```

```
WHERE (
    MEMBERSHIP_START_DATE IS NOT NULL
    AND MEMBERSHIP_START_DATE <> ''
    AND STR_TO_DATE(MEMBERSHIP_START_DATE, '%Y-%m-%d') IS NULL
    AND STR_TO_DATE(MEMBERSHIP_START_DATE, '%d-%m-%Y') IS NULL
)
OR (
    MEMBERSHIP_END_DATE IS NOT NULL
    AND MEMBERSHIP_END_DATE <> ''
    AND STR_TO_DATE(MEMBERSHIP_END_DATE, '%Y-%m-%d') IS NULL
    AND STR_TO_DATE(MEMBERSHIP_END_DATE, '%d-%m-%Y') IS NULL
);
```

RAW_ID	MEMBER_CODE	MEMBERSHIP_START_DATE	MEMBERSHIP_END_DATE
9	M009	2026-02-30	2027-02-28
NULL	NULL	NULL	NULL

```
-- Check invalid book numbers
SELECT *
FROM BOOK_RAW
WHERE PUBLICATION_YEAR NOT REGEXP '^[0-9]{4}$'
   OR COPIES_TOTAL NOT REGEXP '^[0-9]+$'
   OR COPIES_AVAILABLE NOT REGEXP '^[0-9]+$';
```

RAW_ID	BOOK_CODE	ISBN_CODE	BOOK_TITLE	CATEGORY_NAME	PUBLISHER_NAME	AUTHOR_NAME	PUBLICATION_YEAR	B
3	B003	9780553380163	A Brief History of Time	Science	Penguin Random House	Stephen Hawking	1988	2r
4	B004	9780062316097	Sapiens	History	HarperCollins	Yuval Noah Harari	Twenty Fifteen	1s
6	B006	9781612680194	Rich Dad Poor Dad	Business	HarperCollins	Robert Kiyosaki	1997	1s
14	B014	9780000000000	Unknown Book	Sci-Fi	Unknown Publisher	Unknown Author	2005	Fir
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NU

```
-- Check invalid payment amounts and dates
SELECT *
FROM FINE_PAYMENT_RAW
WHERE PAYMENT_AMOUNT IS NULL
   OR TRIM(PAYMENT_AMOUNT) = ''
   OR PAYMENT_AMOUNT NOT REGEXP '^[0-9]+([.][0-9]+)?$';
```

RAW_ID	PAYMENT_CODE	MEMBER_NAME	PAYMENT_DATE	PAYMENT_AMOUNT	PAYMENT_METHOD	TEMP_NOTES
5	FP05	Neha Shah	2026-09-22	-100	Cash	Negative payment
6	FP06	Priya Patel	2026-09-23	One Hundred	UPI	Invalid amount
8	FP08	Karan Malhotra	2026-09-24	NULL	UPI	Missing amount
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
SELECT RAW_ID, PAYMENT_CODE, PAYMENT_DATE
FROM FINE_PAYMENT_RAW
```

```
WHERE PAYMENT_DATE IS NOT NULL
  AND PAYMENT_DATE <> ''
  AND STR_TO_DATE(PAYMENT_DATE, '%Y-%m-%d') IS NULL
  AND STR_TO_DATE(PAYMENT_DATE, '%d/%m/%Y') IS NULL;
```

	RAW_ID	PAYMENT_CODE	PAYMENT_DATE
▶	10	FP10	2026-02-30
*	NULL	NULL	NULL

```
-- Standardize valid date formats
UPDATE MEMBERS_RAW
SET DATE_OF_BIRTH = CASE
    WHEN DATE_OF_BIRTH REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2}$'
        AND SUBSTRING(DATE_OF_BIRTH, 6, 2) BETWEEN '01' AND '12'
        AND SUBSTRING(DATE_OF_BIRTH, 9, 2) BETWEEN '01' AND '31'
        THEN DATE_OF_BIRTH

    WHEN DATE_OF_BIRTH REGEXP '^[0-9]{2}/[0-9]{2}/[0-9]{4}$'
        AND SUBSTRING(DATE_OF_BIRTH, 4, 2) BETWEEN '01' AND '12'
        AND SUBSTRING(DATE_OF_BIRTH, 1, 2) BETWEEN '01' AND '31'
        THEN CONCAT(
            SUBSTRING(DATE_OF_BIRTH, 7, 4), '-',
            SUBSTRING(DATE_OF_BIRTH, 4, 2), '-',
            SUBSTRING(DATE_OF_BIRTH, 1, 2)
        )

    WHEN DATE_OF_BIRTH REGEXP '^[0-9]{2}-[0-9]{2}-[0-9]{4}$'
        AND SUBSTRING(DATE_OF_BIRTH, 4, 2) BETWEEN '01' AND '12'
        AND SUBSTRING(DATE_OF_BIRTH, 1, 2) BETWEEN '01' AND '31'
        THEN CONCAT(
            SUBSTRING(DATE_OF_BIRTH, 7, 4), '-',
            SUBSTRING(DATE_OF_BIRTH, 4, 2), '-',
            SUBSTRING(DATE_OF_BIRTH, 1, 2)
        )

    ELSE NULL
END
WHERE DATE_OF_BIRTH IS NOT NULL
  AND TRIM(DATE_OF_BIRTH) <> '';
```

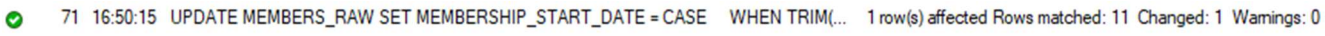
```
69  16:46:27  UPDATE MEMBERS_RAW SET DATE_OF_BIRTH = CASE  WHEN DATE_OF_BIRTH ...  2 row(s) affected Rows matched: 11  Changed: 2  Warnings: 0
```

```
-- Apply the same approach to membership dates:
UPDATE MEMBERS_RAW
SET MEMBERSHIP_START_DATE = CASE
```

```
    WHEN TRIM(MEMBERSHIP_START_DATE) REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2}$'
        AND SUBSTRING(TRIM(MEMBERSHIP_START_DATE), 6, 2) BETWEEN '01' AND
'12'
        AND SUBSTRING(TRIM(MEMBERSHIP_START_DATE), 9, 2) BETWEEN '01' AND
'31'
        THEN TRIM(MEMBERSHIP_START_DATE)

    WHEN TRIM(MEMBERSHIP_START_DATE) REGEXP '^[0-9]{2}-[0-9]{2}-[0-9]{4}$'
        AND SUBSTRING(TRIM(MEMBERSHIP_START_DATE), 4, 2) BETWEEN '01' AND
'12'
        AND SUBSTRING(TRIM(MEMBERSHIP_START_DATE), 1, 2) BETWEEN '01' AND
'31'
        THEN CONCAT(
            SUBSTRING(TRIM(MEMBERSHIP_START_DATE), 7, 4), '-',
            SUBSTRING(TRIM(MEMBERSHIP_START_DATE), 4, 2), '-',
            SUBSTRING(TRIM(MEMBERSHIP_START_DATE), 1, 2)
        )

    ELSE NULL
END
WHERE MEMBERSHIP_START_DATE IS NOT NULL
  AND TRIM(MEMBERSHIP_START_DATE) <> '';
```



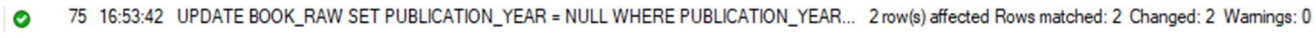

```
-- Normalize payment dates:
UPDATE FINE_PAYMENT_RAW
SET PAYMENT_DATE = CASE
    WHEN TRIM(PAYMENT_DATE) REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2}$'
        AND SUBSTRING(TRIM(PAYMENT_DATE), 6, 2) BETWEEN '01' AND '12'
        AND SUBSTRING(TRIM(PAYMENT_DATE), 9, 2) BETWEEN '01' AND '31'
        THEN TRIM(PAYMENT_DATE)

    WHEN TRIM(PAYMENT_DATE) REGEXP '^[0-9]{2}/[0-9]{2}/[0-9]{4}$'
        AND SUBSTRING(TRIM(PAYMENT_DATE), 4, 2) BETWEEN '01' AND '12'
        AND SUBSTRING(TRIM(PAYMENT_DATE), 1, 2) BETWEEN '01' AND '31'
        THEN CONCAT(
            SUBSTRING(TRIM(PAYMENT_DATE), 7, 4), '-',
            SUBSTRING(TRIM(PAYMENT_DATE), 4, 2), '-',
            SUBSTRING(TRIM(PAYMENT_DATE), 1, 2)
        )
```

```
    ELSE NULL
END
WHERE PAYMENT_DATE IS NOT NULL
  AND TRIM(PAYMENT_DATE) <> '';
```



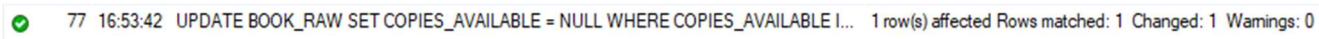

```
-- Convert columns to appropriate data types
UPDATE BOOK_RAW
SET PUBLICATION_YEAR = NULL
WHERE PUBLICATION_YEAR IS NOT NULL
  AND PUBLICATION_YEAR NOT REGEXP '^[0-9]{4}$';
```



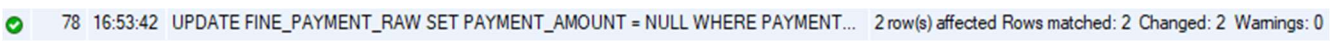

```
UPDATE BOOK_RAW
SET COPIES_TOTAL = NULL
WHERE COPIES_TOTAL IS NOT NULL
  AND COPIES_TOTAL NOT REGEXP '^[0-9]+$';
```




```
UPDATE BOOK_RAW
SET COPIES_AVAILABLE = NULL
WHERE COPIES_AVAILABLE IS NOT NULL
  AND COPIES_AVAILABLE NOT REGEXP '^[0-9]+$';
```




```
UPDATE FINE_PAYMENT_RAW
SET PAYMENT_AMOUNT = NULL
WHERE PAYMENT_AMOUNT IS NOT NULL
  AND PAYMENT_AMOUNT NOT REGEXP '^[0-9]+([.][0-9]+)?$';
```




```
-- convert the columns:
UPDATE MEMBERS_RAW
SET MEMBERSHIP_END_DATE = CONCAT(
    SUBSTRING(MEMBERSHIP_END_DATE, 7, 4), '-',
```

```
    SUBSTRING(MEMBERSHIP_END_DATE, 4, 2), '-',
    SUBSTRING(MEMBERSHIP_END_DATE, 1, 2)
)
WHERE MEMBERSHIP_END_DATE REGEXP '^[0-9]{2}-[0-9]{2}-[0-9]{4}$';
```

81 17:46:06 UPDATE MEMBERS_RAW SET MEMBERSHIP_END_DATE = CONCAT(SUBSTRING(... 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

```
UPDATE MEMBERS_RAW
SET MEMBERSHIP_START_DATE = NULL
WHERE RAW_ID = 9
  AND MEMBERSHIP_START_DATE = '2026-02-30';
```

87 17:51:44 UPDATE MEMBERS_RAW SET MEMBERSHIP_START_DATE = NULL WHERE RAW_I... 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

```
ALTER TABLE MEMBERS_RAW
MODIFY DATE_OF_BIRTH DATE,
MODIFY MEMBERSHIP_START_DATE DATE,
MODIFY MEMBERSHIP_END_DATE DATE;
```

88 17:51:53 ALTER TABLE MEMBERS_RAW MODIFY DATE_OF_BIRTH DATE, MODIFY MEMBERS... 11 row(s) affected Records: 11 Duplicates: 0 Warnings: 0

```
ALTER TABLE BOOK_RAW
MODIFY PUBLICATION_YEAR INT,
MODIFY COPIES_TOTAL INT,
MODIFY COPIES_AVAILABLE INT;
```

89 17:52:24 ALTER TABLE BOOK_RAW MODIFY PUBLICATION_YEAR INT, MODIFY COPIES_TOTA... 13 row(s) affected Records: 13 Duplicates: 0 Warnings: 0

```
UPDATE FINE_PAYMENT_RAW
SET PAYMENT_DATE = NULL
WHERE RAW_ID = 10
  AND PAYMENT_DATE = '2026-02-30';
```

92 17:55:40 UPDATE FINE_PAYMENT_RAW SET PAYMENT_DATE = NULL WHERE RAW_ID = 10 ... 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

```
ALTER TABLE FINE_PAYMENT_RAW
MODIFY PAYMENT_DATE DATE,
MODIFY PAYMENT_AMOUNT DECIMAL(10,2);
```

93 17:56:38 ALTER TABLE FINE_PAYMENT_RAW MODIFY PAYMENT_DATE DATE, MODIFY PAYM... 10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

```
-- Standardize membership account types
-- inspect the distinct values:
SELECT DISTINCT ACCOUNT_TYPE
FROM MEMBERS_RAW;
```

ACCOUNT_TYPE
Student
Faculty
General

```
-- Standardize case and surrounding spaces:
UPDATE MEMBERS_RAW
SET ACCOUNT_TYPE = CASE
    WHEN LOWER(TRIM(ACCOUNT_TYPE)) = 'student'
        THEN 'Student'
    WHEN LOWER(TRIM(ACCOUNT_TYPE)) = 'faculty'
        THEN 'Faculty'
    WHEN LOWER(TRIM(ACCOUNT_TYPE)) = 'general'
        THEN 'General'
    ELSE TRIM(ACCOUNT_TYPE)
END;
```

95 18:02:08 UPDATE MEMBERS_RAW SET ACCOUNT_TYPE = CASE WHEN LOWER(TRIM(ACC... 2 row(s) affected Rows matched: 11 Changed: 2 Warnings: 0

```
-- Standardize membership status
SELECT DISTINCT ACCOUNT_STATUS
FROM MEMBERS_RAW;
```

ACCOUNT_STATUS
Active
Expired
Unactive

```
UPDATE MEMBERS_RAW
SET ACCOUNT_STATUS = CASE
    WHEN LOWER(TRIM(ACCOUNT_STATUS)) = 'active'
        THEN 'Active'
    WHEN LOWER(TRIM(ACCOUNT_STATUS)) IN ('unactive', 'inactive')
        THEN 'Inactive'
    WHEN LOWER(TRIM(ACCOUNT_STATUS)) = 'expired'
        THEN 'Expired'
```

```
    ELSE TRIM(ACCOUNT_STATUS)
END;
```

```
97  18:02:30  UPDATE MEMBERS_RAW SET ACCOUNT_STATUS = CASE  WHEN LOWER(TRIM(A...  4 row(s) affected Rows matched: 11  Changed: 4  Warnings: 0
```

```
-- Standardize book categories
SELECT DISTINCT CATEGORY_NAME
FROM BOOK_RAW;
```

CATEGORY_NAME
Fiction
Science
History
Biography
Business
Literature
Technology
Sci-Fi

```
UPDATE BOOK_RAW
SET CATEGORY_NAME = CASE
    WHEN LOWER(TRIM(CATEGORY_NAME)) = 'fiction'
        THEN 'Fiction'
    WHEN LOWER(TRIM(CATEGORY_NAME)) = 'science'
        THEN 'Science'
    WHEN LOWER(TRIM(CATEGORY_NAME)) = 'history'
        THEN 'History'
    WHEN LOWER(TRIM(CATEGORY_NAME)) = 'technology'
        THEN 'Technology'
    WHEN LOWER(TRIM(CATEGORY_NAME)) = 'biography'
        THEN 'Biography'
    WHEN LOWER(TRIM(CATEGORY_NAME)) = 'business'
        THEN 'Business'
    WHEN LOWER(TRIM(CATEGORY_NAME)) = 'literature'
        THEN 'Literature'
    WHEN LOWER(TRIM(CATEGORY_NAME)) = 'mathematics'
        THEN 'Mathematics'
    ELSE TRIM(CATEGORY_NAME)
END;
```

```
99  18:06:21  UPDATE BOOK_RAW SET CATEGORY_NAME = CASE  WHEN LOWER(TRIM(CATEG...  1 row(s) affected Rows matched: 13  Changed: 1  Warnings: 0
```

```
-- Standardize payment methods
SELECT DISTINCT PAYMENT_METHOD
FROM FINE_PAYMENT_RAW;
```

	PAYMENT_METHOD
▶	Cash
	UPI
	Card
	NULL

```
UPDATE FINE_PAYMENT_RAW
SET PAYMENT_METHOD = CASE
    WHEN LOWER(TRIM(PAYMENT_METHOD)) = 'upi'
        THEN 'UPI'
    WHEN LOWER(TRIM(PAYMENT_METHOD)) = 'cash'
        THEN 'Cash'
    WHEN LOWER(TRIM(PAYMENT_METHOD)) = 'card'
        THEN 'Card'
    ELSE TRIM(PAYMENT_METHOD)
END;
```

101 18:07:18 UPDATE FINE_PAYMENT_RAW SET PAYMENT_METHOD = CASE WHEN LOWER(T... 1 row(s) affected Rows matched: 10 Changed: 1 Warnings: 0

```
-- Check invalid inventory quantities
SELECT *
FROM BOOK_RAW
WHERE COPIES_TOTAL < 0
   OR COPIES_AVAILABLE < 0
   OR COPIES_AVAILABLE > COPIES_TOTAL;
```

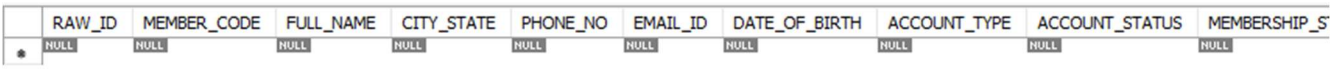
	RAW_ID	BOOK_CODE	ISBN_CODE	BOOK_TITLE	CATEGORY_NAME	PUBLISHER_NAME	AUTHOR_NAME	PUBLICATION_YEAR	BOOK_EDITION
▶	5	B005	9788184001524	Wings of Fire	Biography	Rupa Publications	A.P.J. Abdul Kalam	1999	1st Edition
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
-- Check negative fine payments
SELECT *
FROM FINE_PAYMENT_RAW
WHERE PAYMENT_AMOUNT < 0;
```

	RAW_ID	PAYMENT_CODE	MEMBER_NAME	PAYMENT_DATE	PAYMENT_AMOUNT	PAYMENT_METHOD	TEMP_NOTES
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

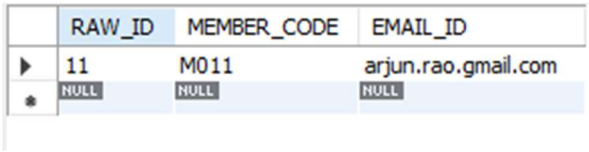
```
-- Check membership date consistency
SELECT *
```

```
FROM MEMBERS_RAW
WHERE MEMBERSHIP_START_DATE IS NOT NULL
  AND MEMBERSHIP_END_DATE IS NOT NULL
  AND MEMBERSHIP_END_DATE < MEMBERSHIP_START_DATE;
```



RAW_ID	MEMBER_CODE	FULL_NAME	CITY_STATE	PHONE_NO	EMAIL_ID	DATE_OF_BIRTH	ACCOUNT_TYPE	ACCOUNT_STATUS	MEMBERSHIP_S
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
-- Check invalid email formats
SELECT RAW_ID, MEMBER_CODE, EMAIL_ID
FROM MEMBERS_RAW
WHERE EMAIL_ID IS NOT NULL
  AND TRIM(EMAIL_ID) <> ''
  AND EMAIL_ID NOT REGEXP
      '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+[.][A-Za-z]{2,}$';
```



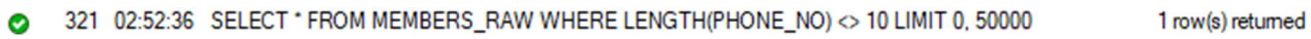
RAW_ID	MEMBER_CODE	EMAIL_ID
11	M011	arjun.rao.gmail.com
NULL	NULL	NULL

```
UPDATE MEMBERS_RAW
SET EMAIL_ID = 'Invalid Email'
WHERE EMAIL_ID NOT REGEXP
      '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+[.][A-Za-z]{2,}$';
```



319 02:46:59 UPDATE MEMBERS_RAW SET EMAIL_ID = 'Invalid Email' WHERE EMAIL_ID NOT REGEXP '^... 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

```
-- INVALID PHONE
SELECT *
FROM MEMBERS_RAW
WHERE LENGTH(PHONE_NO) <> 10;
```



321 02:52:36 SELECT * FROM MEMBERS_RAW WHERE LENGTH(PHONE_NO) <> 10 LIMIT 0, 50000 1 row(s) returned

```
UPDATE MEMBERS_RAW
SET PHONE_NO = 'Invalid No.'
WHERE LENGTH(PHONE_NO) <> 10;
```



322 02:53:45 UPDATE MEMBERS_RAW SET PHONE_NO = 'Invalid No.' WHERE LENGTH(PHONE_NO) <> 10 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

```
SELECT RAW_ID, MEMBER_CODE, FULL_NAME, PHONE_NO
FROM MEMBERS_RAW
```

```
WHERE PHONE_NO IS NOT NULL
      AND PHONE_NO <> 'Invalid No.'
  AND TRIM(PHONE_NO) <> ''
  AND PHONE_NO NOT REGEXP '^[0-9]{10}$';
```

	RAW_ID	MEMBER_CODE	FULL_NAME	PHONE_NO
▶	10	M010	Meera Iyer	98765ABC10
*	NULL	NULL	NULL	NULL

```
UPDATE MEMBERS_RAW
SET PHONE_NO = 'Invalid No.'
WHERE TRIM(PHONE_NO) <> ''
  AND PHONE_NO NOT REGEXP '^[0-9]{10}$';
```

```
327  03:02:44  UPDATE MEMBERS_RAW SET PHONE_NO = 'Invalid No.' WHERE TRIM(PHONE_NO) <> ''  AND...  1 row(s) affected Rows matched: 2  Changed: 1  Warnings: 0
```

```
-- Clean member text columns
UPDATE MEMBERS_RAW
SET
    MEMBER_CODE = TRIM(MEMBER_CODE),
    FULL_NAME = NULLIF(TRIM(FULL_NAME), ''),
    CITY_STATE = NULLIF(TRIM(CITY_STATE), ''),
    PHONE_NO = NULLIF(TRIM(PHONE_NO), ''),
    EMAIL_ID = NULLIF(TRIM(EMAIL_ID), ''),
    ACCOUNT_TYPE = NULLIF(TRIM(ACCOUNT_TYPE), ''),
    ACCOUNT_STATUS = NULLIF(TRIM(ACCOUNT_STATUS), '');
```

```
106  18:11:03  UPDATE MEMBERS_RAW SET  MEMBER_CODE = TRIM(MEMBER_CODE),  FULL_...  4 row(s) affected Rows matched: 11  Changed: 4  Warnings: 0
```

```
-- Standardize email case:
UPDATE MEMBERS_RAW
SET EMAIL_ID = LOWER(EMAIL_ID)
WHERE EMAIL_ID IS NOT NULL;
```

```
107  18:11:03  UPDATE MEMBERS_RAW SET EMAIL_ID = LOWER(EMAIL_ID) WHERE EMAIL_ID IS N...  0 row(s) affected Rows matched: 10  Changed: 0  Warnings: 0
```

```
-- Clean book text columns
UPDATE BOOK_RAW
SET
    BOOK_CODE = TRIM(BOOK_CODE),
    ISBN_CODE = TRIM(ISBN_CODE),
    BOOK_TITLE = NULLIF(TRIM(BOOK_TITLE), ''),
```

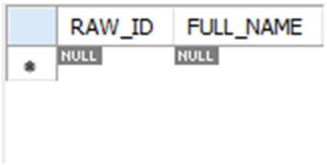
```
    CATEGORY_NAME = NULLIF(TRIM(CATEGORY_NAME), ''),
    PUBLISHER_NAME = NULLIF(TRIM(PUBLISHER_NAME), ''),
    AUTHOR_NAME = NULLIF(TRIM(AUTHOR_NAME), ''),
    BOOK_EDITION = NULLIF(TRIM(BOOK_EDITION), ''),
    SHELF_LOCATION = NULLIF(TRIM(SHELF_LOCATION), '');
```

```
108 18:11:03 UPDATE BOOK_RAW SET BOOK_CODE = TRIM(BOOK_CODE), ISBN_CODE = TRI... 1 row(s) affected Rows matched: 13 Changed: 1 Warnings: 0
```

```
-- Clean fine payment text columns
UPDATE FINE_PAYMENT_RAW
SET
    PAYMENT_CODE = TRIM(PAYMENT_CODE),
    MEMBER_NAME = NULLIF(TRIM(MEMBER_NAME), ''),
    PAYMENT_METHOD = NULLIF(TRIM(PAYMENT_METHOD), '');
```

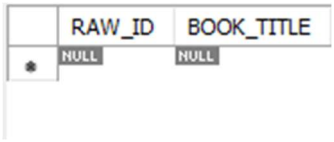
```
109 18:11:03 UPDATE FINE_PAYMENT_RAW SET PAYMENT_CODE = TRIM(PAYMENT_CODE), ... 0 row(s) affected Rows matched: 10 Changed: 0 Warnings: 0
```

```
-- Find repeated internal spaces
SELECT RAW_ID, FULL_NAME
FROM MEMBERS_RAW
WHERE FULL_NAME REGEXP '[[:space:]][[:space:]]+';
```



	RAW_ID	FULL_NAME
*	NULL	NULL

```
SELECT RAW_ID, BOOK_TITLE
FROM BOOK_RAW
WHERE BOOK_TITLE REGEXP '[[:space:]][[:space:]]+';
```



	RAW_ID	BOOK_TITLE
*	NULL	NULL

```
-- Find missing member information
SELECT *
FROM MEMBERS_RAW
```

```
WHERE FULL_NAME IS NULL
   OR CITY_STATE IS NULL
   OR PHONE_NO IS NULL
   OR EMAIL_ID IS NULL
   OR DATE_OF_BIRTH IS NULL
   OR ACCOUNT_TYPE IS NULL
   OR ACCOUNT_STATUS IS NULL;
```

RAW_ID	MEMBER_CODE	FULL_NAME	CITY_STATE	PHONE_NO	EMAIL_ID	DATE_OF_BIRTH	ACCOUNT_TYPE	ACC
6	M006	Neha Shah	Mumbai, Maharashtra	NULL	neha.shah@gmail.com	1995-07-12	General	Inact
8	M008	Anjali Gupta	Mumbai, Maharashtra	NULL	anjali.gupta@gmail.com	2001-09-05	Student	Activ
9	M009	Karan Malhotra	Pune, Maharashtra	9876543218	karan.malhotra@gmail.com	NULL	Student	Activ
12	M012	NULL	Mumbai, Maharashtra	9876543220	NULL	2000-10-10	General	Activ
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
-- Find missing book information
SELECT *
FROM BOOK_RAW
WHERE ISBN_CODE IS NULL
   OR BOOK_TITLE IS NULL
   OR CATEGORY_NAME IS NULL
   OR PUBLISHER_NAME IS NULL
   OR AUTHOR_NAME IS NULL
   OR PUBLICATION_YEAR IS NULL
   OR COPIES_TOTAL IS NULL
   OR COPIES_AVAILABLE IS NULL
   OR SHELF_LOCATION IS NULL;
```

RAW_ID	BOOK_CODE	ISBN_CODE	BOOK_TITLE	CATEGORY_NAME	PUBLISHER_NAME	AUTHOR_NAME	PUBLICATION_YE
3	B003	9780553380163	A Brief History of Time	Science	Penguin Random House	Stephen Hawking	1988
4	B004	9780062316097	Sapiens	History	HarperCollins	Yuval Noah Harari	NULL
6	B006	9781612680194	Rich Dad Poor Dad	Business	HarperCollins	Robert Kiyosaki	1997
10	B010	9780131103627	The C Programming Language	Technology	Pearson Education	Charles Dickens	1988
14	B014	9780000000000	Unknown Book	Sci-Fi	Unknown Publisher	Unknown Author	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
-- Find missing payment information
SELECT *
FROM FINE_PAYMENT_RAW
WHERE MEMBER_NAME IS NULL
   OR PAYMENT_DATE IS NULL
   OR PAYMENT_AMOUNT IS NULL
   OR PAYMENT_METHOD IS NULL;
```

RAW_ID	PAYMENT_CODE	MEMBER_NAME	PAYMENT_DATE	PAYMENT_AMOUNT	PAYMENT_METHOD	TEMP_NOTES
5	FP05	Neha Shah	2026-09-22	NULL	Cash	Negative payment
6	FP06	Priya Patel	2026-09-23	NULL	UPI	Invalid amount
8	FP08	Karan Malhotra	2026-09-24	NULL	UPI	Missing amount
9	FP09	Meera Iyer	2026-09-25	200.00	NULL	Missing payment method
10	FP10	Anjali Gupta	NULL	150.00	Cash	Invalid date
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
-- Convert empty strings to NULL
UPDATE MEMBERS_RAW
SET
    FULL_NAME = NULLIF(FULL_NAME, ''),
    CITY_STATE = NULLIF(CITY_STATE, ''),
    PHONE_NO = NULLIF(PHONE_NO, ''),
    EMAIL_ID = NULLIF(EMAIL_ID, ''),
    ACCOUNT_TYPE = NULLIF(ACCOUNT_TYPE, ''),
    ACCOUNT_STATUS = NULLIF(ACCOUNT_STATUS, '');
```

```
117  18:17:42  UPDATE MEMBERS_RAW SET  FULL_NAME = NULLIF(FULL_NAME, ''),  CITY_STA...  0 row(s) affected Rows matched: 11  Changed: 0  Warnings: 0
```

```
UPDATE BOOK_RAW
SET
    ISBN_CODE = NULLIF(ISBN_CODE, ''),
    BOOK_TITLE = NULLIF(BOOK_TITLE, ''),
    CATEGORY_NAME = NULLIF(CATEGORY_NAME, ''),
    PUBLISHER_NAME = NULLIF(PUBLISHER_NAME, ''),
    AUTHOR_NAME = NULLIF(AUTHOR_NAME, ''),
    BOOK_EDITION = NULLIF(BOOK_EDITION, ''),
    SHELF_LOCATION = NULLIF(SHELF_LOCATION, '');
```

```
118  18:17:42  UPDATE BOOK_RAW SET  ISBN_CODE = NULLIF(ISBN_CODE, ''),  BOOK_TITLE = ...  0 row(s) affected Rows matched: 13  Changed: 0  Warnings: 0
```

```
UPDATE FINE_PAYMENT_RAW
SET
    MEMBER_NAME = NULLIF(MEMBER_NAME, ''),
    PAYMENT_METHOD = NULLIF(PAYMENT_METHOD, '');
```

```
119  18:17:42  UPDATE FINE_PAYMENT_RAW SET  MEMBER_NAME = NULLIF(MEMBER_NAME, ''),...  0 row(s) affected Rows matched: 10  Changed: 0  Warnings: 0
```

```
-- Check missing-value counts
SELECT
    COUNT(*) AS TOTAL_RECORDS,
    SUM(FULL_NAME IS NULL) AS MISSING_NAMES,
    SUM(CITY_STATE IS NULL) AS MISSING_CITY_STATE,
```

```
    SUM(PHONE_NO IS NULL) AS MISSING_PHONE,
    SUM(EMAIL_ID IS NULL) AS MISSING_EMAIL,
    SUM(DATE_OF_BIRTH IS NULL) AS MISSING_DOB
FROM MEMBERS_RAW;
```

	TOTAL_RECORDS	MISSING_NAMES	MISSING_CITY_STATE	MISSING_PHONE	MISSING_EMAIL	MISSING_DOB
▶	11	1	0	2	1	1

```
SELECT
    COUNT(*) AS TOTAL_RECORDS,
    SUM(ISBN_CODE IS NULL) AS MISSING_ISBN,
    SUM(PUBLICATION_YEAR IS NULL) AS MISSING_YEAR,
    SUM(COPIES_TOTAL IS NULL) AS MISSING_TOTAL_COPIES,
    SUM(COPIES_AVAILABLE IS NULL) AS MISSING_AVAILABLE_COPIES
FROM BOOK_RAW;
```

	TOTAL_RECORDS	MISSING_ISBN	MISSING_YEAR	MISSING_TOTAL_COPIES	MISSING_AVAILABLE_COPIES
▶	13	0	2	3	2

```
SELECT
    COUNT(*) AS TOTAL_RECORDS,
    SUM(PAYMENT_DATE IS NULL) AS MISSING_PAYMENT_DATE,
    SUM(PAYMENT_AMOUNT IS NULL) AS MISSING_AMOUNT,
    SUM(PAYMENT_METHOD IS NULL) AS MISSING_METHOD
FROM FINE_PAYMENT_RAW;
```

	TOTAL_RECORDS	MISSING_PAYMENT_DATE	MISSING_AMOUNT	MISSING_METHOD
▶	10	1	3	1

1. SELECT,ALTER AND CRUD OPERATIONS

DISPLAY ALL THE INFO IN BOOK TABLE

```
SELECT * FROM BOOK;
```

BOOK_ID	ISBN_CODE	BOOK_TITLE	CATEGORY_ID	PUBLISHER_ID	PUBLICATION_YEAR	BOOK_EDITION	COPIES_TC
B001	9780451524935	1984	C001	P001	1949	1st Edition	10
B002	9780747532699	Harry Potter and the Philosophers Stone	C001	P003	1997	1st Edition	12
B003	9780553380163	A Brief History of Time	C002	P001	1988	2nd Edition	8
B004	9780062316097	Sapiens	C003	P003	2015	1st Edition	10
B005	9788184001524	Wings of Fire	C005	P007	1999	1st Edition	15
B006	9781612680194	Rich Dad Poor Dad	C006	P003	1997	1st Edition	10
B007	9788184000713	The Guide	C007	P007	1958	1st Edition	7
B008	9780262033848	Introduction to Algorithms	C004	P004	2009	3rd Edition	8
B009	9780132350884	Clean Code	C004	P004	2008	1st Edition	10
B010	9780131103627	The C Programming Language	C004	P004	1988	2nd Edition	8
B011	9780199535569	Great Expectations	C007	P002	1861	2nd Edition	6
B012	9780061120084	The Alchemist	C001	P003	1988	1st Edition	10
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

DISPLAY SPECIFIC INFO FROM BOOK TABLE

```
SELECT BOOK_TITLE,
      ISBN_CODE,
    PUBLICATION_YEAR,
    COPIES_TOTAL,
    COPIES_AVAILABLE
FROM BOOK;
```

BOOK_TITLE	ISBN_CODE	PUBLICATION_YEAR	COPIES_TOTAL	COPIES_AVAILABLE
1984	9780451524935	1949	10	7
Harry Potter and the Philosophers Stone	9780747532699	1997	12	10
A Brief History of Time	9780553380163	1988	8	6
Sapiens	9780062316097	2015	10	8
Wings of Fire	9788184001524	1999	15	12
Rich Dad Poor Dad	9781612680194	1997	10	8
The Guide	9788184000713	1958	7	5
Introduction to Algorithms	9780262033848	2009	8	6
Clean Code	9780132350884	2008	10	8
The C Programming Language	9780131103627	1988	8	6
Great Expectations	9780199535569	1861	6	4
The Alchemist	9780061120084	1988	10	8

DISPLAY FIRST NAME, LAST NAME, CITY AND EMAIL FROM MEMBERS TABLE

```
SELECT FIRST_NAME,
      LAST_NAME,
    CITY,
    EMAIL_ID
FROM MEMBERS;
```

	FIRST_NAME	LAST_NAME	CITY	EMAIL_ID
▶	Rahul	Sharma	Mumbai	rahul.sharma@gmail.com
	Priya	Patel	Mumbai	priya.patel@gmail.com
	Amit	Joshi	Thane	amit.joshi@gmail.com
	Sneha	Desai	Mumbai	sneha.desai@gmail.com
	Vikas	Mehta	Navi Mumbai	vikas.mehta@gmail.com
	Neha	Shah	Mumbai	neha.shah@gmail.com
	Rajesh	Kulkarni	Pune	rajesh.kulkarni@gmail.com
	Anjali	Gupta	Mumbai	anjali.gupta@gmail.com

DISPLAY FIRST NAME, LAST NAME FROM AUTHORS TABLE

```
SELECT FIRST_NAME,
      LAST_NAME
FROM AUTHOR;
```

	FIRST_NAME	LAST_NAME
▶	George	Orwell
	J.K.	Rowling
	Stephen	Hawking
	Yuval Noah	Harari
	R.K.	Narayan
	Robert	Kiyosaki
	A.P.J. Abdul	Kalam
	Chetan	Bhagat
	Isaac	Asimov
	Charles	Dickens

```
--  Track book condition
ALTER TABLE BOOK
ADD BOOK_CONDITION VARCHAR(30);
```




```
--  Segment members by occupation
ALTER TABLE MEMBERS
ADD OCCUPATION VARCHAR(100);
```

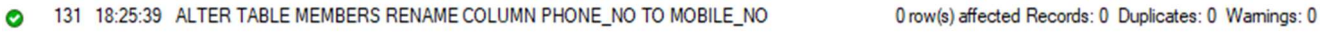



```
--  Accommodate different book edition formats
ALTER TABLE BOOK
```

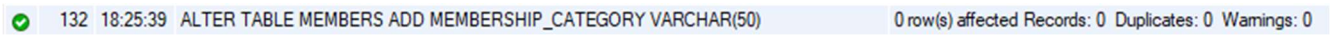
```
MODIFY BOOK_EDITION VARCHAR(50);
```



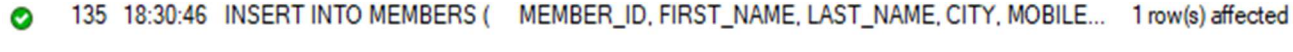

```
--  Improve member contact column naming
ALTER TABLE MEMBERS
RENAME COLUMN PHONE_NO TO MOBILE_NO;
```




```
-- Support membership category analysis
ALTER TABLE MEMBERS
ADD MEMBERSHIP_CATEGORY VARCHAR(50);
```

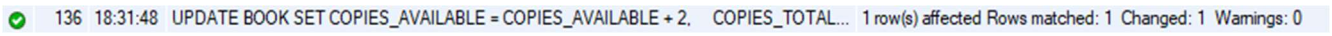



```
-- Register a new library member.
INSERT INTO MEMBERS (
    MEMBER_ID, FIRST_NAME, LAST_NAME, CITY, MOBILE_NO, EMAIL_ID,
DATE_OF_BIRTH, ACTIVE_STATUS_ID, OCCUPATION,
    MEMBERSHIP_CATEGORY
)
VALUES (
    'M009',
    'Rohan',
    'Mehta',
    'Mumbai',
    '9876543218',
    'rohan.mehta@gmail.com',
    '2003-06-15',
    'S001',
    'Student',
    'Regular'
);
```

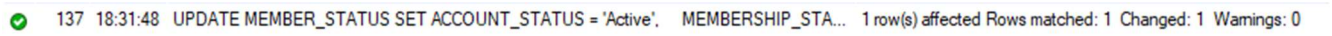



```
-- Adjust inventory after new copies are added.
UPDATE BOOK
SET COPIES_AVAILABLE = COPIES_AVAILABLE + 2,
```

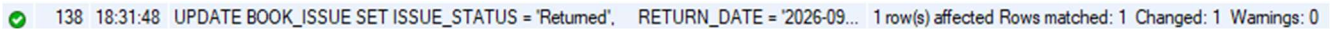
```
    COPIES_TOTAL = COPIES_TOTAL + 2
WHERE BOOK_ID = 'B003';
```



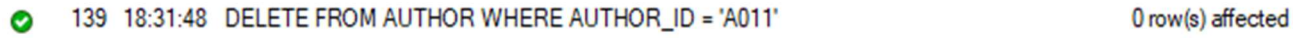

```
-- Renew or update a member's membership period.
UPDATE MEMBER_STATUS
SET ACCOUNT_STATUS = 'Active',
    MEMBERSHIP_START_DATE = '2026-09-27',
    MEMBERSHIP_END_DATE = '2027-09-27'
WHERE ACTIVE_STATUS_ID = 'S001';
```




```
-- Update the issue record when a book is returned.
UPDATE BOOK_ISSUE
SET ISSUE_STATUS = 'Returned',
    RETURN_DATE = '2026-09-27'
WHERE ISSUE_ID = 'I004';
```




```
-- Remove an author record that is no longer required.
DELETE FROM AUTHOR
WHERE AUTHOR_ID = 'A011';
```



2. BUILT-IN FUNCTIONS

String Functions

```
--  Standardize book titles
SELECT
    BOOK_TITLE,
    UPPER(TRIM(BOOK_TITLE)) AS STANDARDIZED_BOOK_TITLE
FROM BOOK;
```

	BOOK_TITLE	STANDARDIZED_BOOK_TITLE
▶	1984	1984
	Harry Potter and the Philosophers Stone	HARRY POTTER AND THE PHILOSOPHERS STONE
	A Brief History of Time	A BRIEF HISTORY OF TIME
	Sapiens	SAPIENS
	Wings of Fire	WINGS OF FIRE
	Rich Dad Poor Dad	RICH DAD POOR DAD
	The Guide	THE GUIDE
	Introduction to Algorithms	INTRODUCTION TO ALGORITHMS
	Clean Code	CLEAN CODE
	The C Programming Language	THE C PROGRAMMING LANGUAGE
	Great Expectations	GREAT EXPECTATIONS
	The Alchemist	THE ALCHEMIST

```
--  Identify members with missing email addresses
SELECT
    FIRST_NAME,
    LAST_NAME,
    EMAIL_ID,
    IFNULL(EMAIL_ID, 'No Email') AS EMAIL_STATUS
FROM MEMBERS;
```

	FIRST_NAME	LAST_NAME	EMAIL_ID	EMAIL_STATUS
▶	Rahul	Sharma	rahul.sharma@gmail.com	rahul.sharma@gmail.com
	Priya	Patel	priya.patel@gmail.com	priya.patel@gmail.com
	Amit	Joshi	amit.joshi@gmail.com	amit.joshi@gmail.com
	Sneha	Desai	sneha.desai@gmail.com	sneha.desai@gmail.com
	Vikas	Mehta	vikas.mehta@gmail.com	vikas.mehta@gmail.com
	Neha	Shah	neha.shah@gmail.com	neha.shah@gmail.com
	Rajesh	Kulkarni	rajesh.kulkarni@gmail.com	rajesh.kulkarni@gmail.com
	Anjali	Gupta	anjali.gupta@gmail.com	anjali.gupta@gmail.com
	Rohan	Mehta	rohan.mehta@gmail.com	rohan.mehta@gmail.com

```
--  Generate full author names
SELECT
    CONCAT(FIRST_NAME, ' ', LAST_NAME) AS FULL_AUTHOR_NAME
FROM AUTHOR;
```

	FULL_AUTHOR_NAME
▶	George Orwell
	J.K. Rowling
	Stephen Hawking
	Yuval Noah Harari
	R.K. Narayan
	Robert Kiyosaki
	A.P.J. Abdul Kalam
	Chetan Bhagat
	Isaac Asimov
	Charles Dickens

```
--  Identify books containing the keyword 'The'
SELECT
    BOOK_TITLE,
    INSTR(LOWER(BOOK_TITLE), 'the') AS KEYWORD_POSITION
FROM BOOK
WHERE INSTR(LOWER(BOOK_TITLE), 'the') > 0;
```

	BOOK_TITLE	KEYWORD_POSITION
▶	Harry Potter and the Philosophers Stone	18
	The Guide	1
	The C Programming Language	1
	The Alchemist	1

```
--  Generate member details for reporting
SELECT
    CONCAT(FIRST_NAME, ' ', LAST_NAME) AS MEMBER_NAME,
    CITY,
    CONCAT(FIRST_NAME, ' ', LAST_NAME, ' - ', CITY) AS MEMBER_DETAILS
FROM MEMBERS;
```

	MEMBER_NAME	CITY	MEMBER_DETAILS
▶	Rahul Sharma	Mumbai	Rahul Sharma - Mumbai
	Priya Patel	Mumbai	Priya Patel - Mumbai
	Amit Joshi	Thane	Amit Joshi - Thane
	Sneha Desai	Mumbai	Sneha Desai - Mumbai
	Vikas Mehta	Navi Mumbai	Vikas Mehta - Navi Mumbai
	Neha Shah	Mumbai	Neha Shah - Mumbai
	Rajesh Kulkarni	Pune	Rajesh Kulkarni - Pune
	Anjali Gupta	Mumbai	Anjali Gupta - Mumbai
	Rohan Mehta	Mumbai	Rohan Mehta - Mumbai

Date Functions

```sql
-- Track overdue book returns
SELECT
    ISSUE_ID,
    ISSUE_DATE,
    RETURN_DATE,
    DATE_ADD(ISSUE_DATE, INTERVAL 7 DAY) AS DUE_DATE,
    DATEDIFF(CURDATE(), DATE_ADD(ISSUE_DATE, INTERVAL 7 DAY))
        AS DAYS_PAST_DUE
FROM BOOK_ISSUE
WHERE RETURN_DATE IS NULL
  AND CURDATE() > DATE_ADD(ISSUE_DATE, INTERVAL 7 DAY);
```

ISSUE_ID	ISSUE_DATE	RETURN_DATE	DUE_DATE	DAYS_PAST_DUE
I005	2026-09-08	NULL	2026-09-15	33
I007	2026-09-10	NULL	2026-09-17	31
I009	2026-09-12	NULL	2026-09-19	29

```sql
-- Calculate book borrowing duration
SELECT
    ISSUE_ID,
    ISSUE_DATE,
    RETURN_DATE,
    DATEDIFF(
        COALESCE(RETURN_DATE, CURDATE()),
        ISSUE_DATE
    ) AS BORROWING_DAYS
FROM BOOK_ISSUE;
```

ISSUE_ID	ISSUE_DATE	RETURN_DATE	BORROWING_DAYS
I001	2026-09-01	2026-09-10	9
I002	2026-09-03	2026-09-12	9
I003	2026-09-05	2026-09-15	10
I004	2026-09-07	2026-09-27	20
I005	2026-09-08	NULL	40
I006	2026-09-09	2026-09-18	9
I007	2026-09-10	NULL	38
I008	2026-09-11	2026-09-20	9
I009	2026-09-12	NULL	36
I010	2026-09-13	2026-09-19	6

```sql
-- Identify memberships expiring within the next 30 days
SELECT
    ACTIVE_STATUS_ID,
    MEMBERSHIP_END_DATE,
    DATEDIFF(MEMBERSHIP_END_DATE, CURDATE()) AS DAYS_TO_EXPIRY
```

```
FROM MEMBER_STATUS
WHERE MEMBERSHIP_END_DATE BETWEEN CURDATE()
      AND DATE_ADD(CURDATE(), INTERVAL 30 DAY);
```

ACTIVE_STATUS_ID	MEMBERSHIP_END_DATE	DAYS_TO_EXPIRY

```
-- Analyze monthly membership registrations
SELECT
    YEAR(MEMBERSHIP_START_DATE) AS REGISTRATION_YEAR,
    MONTHNAME(MEMBERSHIP_START_DATE) AS REGISTRATION_MONTH,
    COUNT(*) AS MEMBERSHIP_COUNT
FROM MEMBER_STATUS
GROUP BY
    YEAR(MEMBERSHIP_START_DATE),
    MONTH(MEMBERSHIP_START_DATE),
    MONTHNAME(MEMBERSHIP_START_DATE)
ORDER BY
    REGISTRATION_YEAR,
    MONTH(MEMBERSHIP_START_DATE);
```

REGISTRATION_YEAR	REGISTRATION_MONTH	MEMBERSHIP_COUNT
2025	January	1
2025	June	1
2026	January	1
2026	February	1
2026	March	1
2026	April	1
2026	May	1
2026	September	1

```
-- Calculate member age
SELECT
    FIRST_NAME,
    LAST_NAME,
    DATE_OF_BIRTH,
    TIMESTAMPDIFF(YEAR, DATE_OF_BIRTH, CURDATE()) AS MEMBER_AGE
FROM MEMBERS
WHERE DATE_OF_BIRTH IS NOT NULL;
```

	FIRST_NAME	LAST_NAME	DATE_OF_BIRTH	MEMBER_AGE
▶	Rahul	Sharma	2003-05-15	23
	Priya	Patel	2002-08-21	24
	Amit	Joshi	1985-02-10	41
	Sneha	Desai	1998-11-25	27
	Vikas	Mehta	2004-01-18	22
	Neha	Shah	1995-07-12	31
	Rajesh	Kulkarni	1978-03-30	48
	Anjali	Gupta	2001-09-05	25
	Rohan	Mehta	2003-06-15	23

Numeric Functions

```
-- Calculate total available book inventory
SELECT
    SUM(COPIES_AVAILABLE) AS TOTAL_AVAILABLE_COPIES
FROM BOOK;
```

	TOTAL_AVAILABLE_COPIES
▶	90

```
-- Calculate the percentage of books available
SELECT
    SUM(COPIES_TOTAL) AS TOTAL_COPIES,
    SUM(COPIES_AVAILABLE) AS AVAILABLE_COPIES,
    ROUND(
        SUM(COPIES_AVAILABLE) * 100.0 /
        NULLIF(SUM(COPIES_TOTAL), 0),
        2
    ) AS AVAILABILITY_PERCENTAGE
FROM BOOK;
```

	TOTAL_COPIES	AVAILABLE_COPIES	AVAILABILITY_PERCENTAGE
▶	116	90	77.59

```
-- Calculate additional fine charges
SELECT
    FINE_ID,
    FINE_TOTAL,
    ROUND(FINE_TOTAL * 1.10, 2) AS FINE_WITH_10_PERCENT_ADDITION
FROM FINE_DUE;
```

	FINE_ID	FINE_TOTAL	FINE_WITH_10_PERCENT_ADDITION
▶	F001	50.00	55.00
	F002	100.00	110.00
	F003	75.00	82.50
	F004	50.00	55.00
	F005	25.00	27.50

```
-- Calculate total outstanding fines
SELECT
    ROUND(SUM(FINE_TOTAL), 2) AS TOTAL_FINE_AMOUNT
FROM FINE_DUE;
```

	TOTAL_FINE_AMOUNT
▶	300.00

```
-- Identify books with the highest number of issued copies
SELECT
    BOOK_TITLE,
    COPIES_TOTAL,
    COPIES_AVAILABLE,
    ABS(COPIES_TOTAL - COPIES_AVAILABLE) AS COPIES_ISSUED
FROM BOOK
ORDER BY COPIES_ISSUED DESC
LIMIT 5;
```

	BOOK_TITLE	COPIES_TOTAL	COPIES_AVAILABLE	COPIES_ISSUED
▶	1984	10	7	3
	Wings of Fire	15	12	3
	Harry Potter and the Philosophers Stone	12	10	2
	A Brief History of Time	10	8	2
	Sapiens	10	8	2

3. FILERING AND SORTING

FILTERING

```
-- Identify books with low availability
-- Help library management identify books that may need additional copies
because stock is running low.
SELECT
    BOOK_TITLE,
```

```
    COPIES_TOTAL,
    COPIES_AVAILABLE
FROM BOOK
WHERE COPIES_AVAILABLE <= 5;
```

BOOK_TITLE	COPIES_TOTAL	COPIES_AVAILABLE
The Guide	7	5
Great Expectations	6	4

```
-- Identify books that have not been returned
-- Track books currently issued to members to support follow-ups and
borrowing management.
SELECT *
FROM BOOK_ISSUE
WHERE ISSUE_STATUS IS NULL
   OR ISSUE_STATUS = 'Issued';
```

ISSUE_ID	BOOK_ID	MEMBER_ID	ISSUE_DATE	RETURN_DATE	ISSUE_STATUS	ISSUED_BY_ID
I005	B008	M001	2026-09-08	NULL	Issued	ST02
I007	B009	M008	2026-09-10	NULL	Issued	ST01
I009	B010	M002	2026-09-12	NULL	Issued	ST02
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
-- Analyze books published between 1990 and 2010
-- Identify books from a specific publication period for collection
analysis and catalog planning.
SELECT
    BOOK_TITLE,
    PUBLICATION_YEAR,
    CATEGORY_ID,
    COPIES_AVAILABLE
FROM BOOK
WHERE PUBLICATION_YEAR BETWEEN 1990 AND 2010;
```

BOOK_TITLE	PUBLICATION_YEAR	CATEGORY_ID	COPIES_AVAILABLE
Harry Potter and the Philosophers Stone	1997	C001	10
Wings of Fire	1999	C005	12
Rich Dad Poor Dad	1997	C006	8
Introduction to Algorithms	2009	C004	6
Clean Code	2008	C004	8

```
-- Identify members from major service cities
```

```
-- Analyze membership across important cities to understand the library's
geographical reach.
SELECT
    MEMBER_ID,
    FIRST_NAME,
    LAST_NAME,
    CITY
FROM MEMBERS
WHERE CITY IN ('Mumbai', 'Pune', 'Thane');
```

MEMBER_ID	FIRST_NAME	LAST_NAME	CITY
M001	Rahul	Sharma	Mumbai
M002	Priya	Patel	Mumbai
M003	Amit	Joshi	Thane
M004	Sneha	Desai	Mumbai
M006	Neha	Shah	Mumbai
M007	Rajesh	Kulkarni	Pune
M008	Anjali	Gupta	Mumbai
M009	Rohan	Mehta	Mumbai
NULL	NULL	NULL	NULL

```
-- Identify high-value outstanding fines
-- Find fines of ₹50 or more for follow-up and payment collection.
SELECT
    FINE_ID,
    MEMBER_ID,
    FINE_TOTAL,
    FINE_DATE
FROM FINE_DUE
WHERE FINE_TOTAL >= 50;
```

FINE_ID	MEMBER_ID	FINE_TOTAL	FINE_DATE
F001	M001	50.00	2026-09-11
F002	M003	100.00	2026-09-16
F003	M007	75.00	2026-09-19
F004	M003	50.00	2026-09-21
NULL	NULL	NULL	NULL

```
-- Find authors whose last name starts with S
SELECT FIRST_NAME,
LAST_NAME FROM AUTHOR
WHERE LAST_NAME LIKE 'S%';
```

	FIRST_NAME	LAST_NAME

```
-- Find members whose first name ends with a
SELECT FIRST_NAME,
LAST_NAME FROM MEMBERS
WHERE FIRST_NAME LIKE '%A';
```

	FIRST_NAME	LAST_NAME
▶	Priya	Patel
	Sneha	Desai
	Neha	Shah

```
-- Find members whose email contains gmail
SELECT FIRST_NAME,
LAST_NAME,
EMAIL_ID FROM MEMBERS
WHERE EMAIL_ID LIKE '%GMAIL%';
```

	FIRST_NAME	LAST_NAME	EMAIL_ID
▶	Rahul	Sharma	rahul.sharma@gmail.com
	Priya	Patel	priya.patel@gmail.com
	Amit	Joshi	amit.joshi@gmail.com
	Sneha	Desai	sneha.desai@gmail.com
	Vikas	Mehta	vikas.mehta@gmail.com
	Neha	Shah	neha.shah@gmail.com
	Rajesh	Kulkarni	rajesh.kulkarni@gmail.com
	Anjali	Gupta	anjali.gupta@gmail.com
	Rohan	Mehta	rohan.mehta@gmail.com

```
-- Find members whose first name has exactly 5 characters.
SELECT FIRST_NAME,
LAST_NAME FROM MEMBERS
WHERE FIRST_NAME LIKE '_____';
```

	FIRST_NAME	LAST_NAME
▶	Rahul	Sharma
	Priya	Patel
	Sneha	Desai
	Vikas	Mehta
	Rohan	Mehta

SORTING

```
-- Identify the category with the most books
-- Determine which book category has the largest collection to support
-- collection planning and resource allocation.
SELECT
    CATEGORY_ID,
    COUNT(*) AS TOTAL_BOOKS
FROM BOOK
GROUP BY CATEGORY_ID
ORDER BY TOTAL_BOOKS DESC
LIMIT 1;
```

	CATEGORY_ID	TOTAL_BOOKS
▶	C001	3

```
-- Identify members who issue more than one book
-- Business scenario: Identify frequent library users and understand
borrowing behavior.
SELECT
    MEMBER_ID,
    COUNT(*) AS TOTAL_ISSUES
FROM BOOK_ISSUE
GROUP BY MEMBER_ID
HAVING COUNT(*) > 1
ORDER BY TOTAL_ISSUES DESC;
```

	MEMBER_ID	TOTAL_ISSUES
▶	M001	2
	M002	2
	M003	2
	M004	2

```
-- Identify members with total fines greater than ₹50
```

```
-- Find members with relatively high accumulated fines for payment follow-
up.
SELECT
    MEMBER_ID,
    ROUND(SUM(FINE_TOTAL), 2) AS TOTAL_FINE
FROM FINE_DUE
GROUP BY MEMBER_ID
HAVING SUM(FINE_TOTAL) > 50
ORDER BY TOTAL_FINE DESC;
```

MEMBER_ID	TOTAL_FINE
M003	150.00
M007	75.00

```
-- Identify books with the highest number of unavailable copies
-- Analyze inventory utilization and identify books with many copies
currently unavailable.
SELECT
    BOOK_TITLE,
    COPIES_TOTAL,
    COPIES_AVAILABLE,
    COPIES_TOTAL - COPIES_AVAILABLE AS UNAVAILABLE_COPIES
FROM BOOK
ORDER BY UNAVAILABLE_COPIES DESC
LIMIT 5;
```

BOOK_TITLE	COPIES_TOTAL	COPIES_AVAILABLE	UNAVAILABLE_COPIES
1984	10	7	3
Wings of Fire	15	12	3
Harry Potter and the Philosophers Stone	12	10	2
A Brief History of Time	10	8	2
Sapiens	10	8	2

```
-- Identify locations storing more than one book
-- Understand how the library's book collection is distributed across
storage locations.
SELECT
    LOCATION_ID,
    COUNT(*) AS TOTAL_BOOKS
FROM BOOK
GROUP BY LOCATION_ID
HAVING COUNT(*) > 1
ORDER BY TOTAL_BOOKS DESC;
```

	LOCATION_ID	TOTAL_BOOKS
▶	L001	3
	L004	3
	L007	2

AGREGATE FUNCTION AND GROUP BY

```
-- Calculate total book copies in each category – SUM(), GROUP BY
-- Analyze inventory distribution across book categories to support
-- purchasing and resource allocation decisions.
SELECT
    CATEGORY_ID,
    SUM(COPIES_TOTAL) AS TOTAL_COPIES
FROM BOOK
GROUP BY CATEGORY_ID
ORDER BY TOTAL_COPIES DESC;
```

	CATEGORY_ID	TOTAL_COPIES
▶	C001	32
	C004	26
	C005	15
	C007	13
	C002	10
	C003	10
	C006	10

```
-- Identify the number of books in each category – COUNT(), GROUP BY
-- Measure the size of each category's collection and identify categories
with fewer books.
SELECT
    CATEGORY_ID,
    COUNT(*) AS TOTAL_BOOKS
FROM BOOK
GROUP BY CATEGORY_ID
ORDER BY TOTAL_BOOKS DESC;
```

	CATEGORY_ID	TOTAL_BOOKS
▶	C001	3
	C004	3
	C007	2
	C002	1
	C003	1
	C005	1
	C006	1

```
-- Calculate the average number of copies per book in each category –
AVG(), GROUP BY
-- Compare inventory levels across categories to understand whether some
-- categories have more copies per book than others.
SELECT
    CATEGORY_ID,
    ROUND(AVG(COPIES_TOTAL), 2) AS AVG_COPIES_PER_BOOK
FROM BOOK
GROUP BY CATEGORY_ID
ORDER BY AVG_COPIES_PER_BOOK DESC;
```

CATEGORY_ID	AVG_COPIES_PER_BOOK
C005	15.00
C001	10.67
C002	10.00
C003	10.00
C006	10.00
C004	8.67
C007	6.50

```
-- Identify the highest and lowest fines for each member – MAX(), MIN(),
GROUP BY
-- Analyze fine patterns for each member to support payment follow-ups and
-- identify unusually high fines.
SELECT
    MEMBER_ID,
    MIN(FINE_TOTAL) AS MIN_FINE,
    MAX(FINE_TOTAL) AS MAX_FINE,
    SUM(FINE_TOTAL) AS TOTAL_FINE,
    ROUND(AVG(FINE_TOTAL), 2) AS AVG_FINE
FROM FINE_DUE
GROUP BY MEMBER_ID
ORDER BY TOTAL_FINE DESC;
```

MEMBER_ID	MIN_FINE	MAX_FINE	TOTAL_FINE	AVG_FINE
M003	50.00	100.00	150.00	75.00
M007	75.00	75.00	75.00	75.00
M001	50.00	50.00	50.00	50.00
M004	25.00	25.00	25.00	25.00

```
-- Analyze borrowing activity by member – COUNT(), GROUP BY
-- Identify frequent borrowers and understand member engagement with
library resources.
SELECT
    MEMBER_ID,
```

```
    COUNT(*) AS TOTAL_ISSUE_RECORDS
FROM BOOK_ISSUE
GROUP BY MEMBER_ID
ORDER BY TOTAL_ISSUE_RECORDS DESC;
```

MEMBER_ID	TOTAL_ISSUE_RECORDS
M001	2
M002	2
M003	2
M004	2
M007	1
M008	1

5. JOINS

```
-- Analyze book borrowing activity – INNER JOIN
-- Analyze which books are borrowed, by which members, and when they are
returned.
SELECT
    B.BOOK_TITLE,
    CONCAT(M.FIRST_NAME, ' ', M.LAST_NAME) AS MEMBER_NAME,
    I.ISSUE_DATE,
    I.RETURN_DATE,
    I.ISSUE_STATUS,
    LS.STAFF_NAME AS ISSUED_BY
FROM BOOK B
INNER JOIN BOOK_ISSUE I
    ON B.BOOK_ID = I.BOOK_ID
INNER JOIN MEMBERS M
    ON I.MEMBER_ID = M.MEMBER_ID
INNER JOIN LIBRARY_STAFF LS
    ON I.ISSUED_BY_ID = LS.ISSUED_BY_ID;
```

BOOK_TITLE	MEMBER_NAME	ISSUE_DATE	RETURN_DATE	ISSUE_STATUS	ISSUED_BY
1984	Rahul Sharma	2026-09-01	2026-09-10	Returned	Rajesh Kumar
Wings of Fire	Amit Joshi	2026-09-05	2026-09-15	Returned	Rajesh Kumar
Clean Code	Anjali Gupta	2026-09-10	NULL	Issued	Rajesh Kumar
A Brief History of Time	Priya Patel	2026-09-03	2026-09-12	Returned	Anita Sharma
Introduction to Algorithms	Rahul Sharma	2026-09-08	NULL	Issued	Anita Sharma
The C Programming Language	Priya Patel	2026-09-12	NULL	Issued	Anita Sharma
Rich Dad Poor Dad	Sneha Desai	2026-09-07	2026-09-27	Returned	Vijay Patil
Sapiens	Amit Joshi	2026-09-11	2026-09-20	Returned	Vijay Patil
Harry Potter and the Philosophers Stone	Rajesh Kulkarni	2026-09-09	2026-09-18	Returned	Meena Joshi
The Alchemist	Sneha Desai	2026-09-13	2026-09-19	Returned	Meena Joshi

```
-- Identify books without publisher information – LEFT JOIN
-- Find all books, including those that do not have a matching publisher
record.
SELECT
    B.BOOK_ID,
    B.BOOK_TITLE,
    P.PUBLISHER_NAME
FROM BOOK B
LEFT JOIN PUBLISHER P
    ON B.PUBLISHER_ID = P.PUBLISHER_ID
WHERE P.PUBLISHER_ID IS NULL;
```

	BOOK_ID	BOOK_TITLE	PUBLISHER_NAME

```
-- Identify library locations without books – RIGHT JOIN
-- Find all library locations, including shelves or locations where no
books are currently recorded.
SELECT
    L.LOCATION_ID,
    L.SHELF_NO,
    L.SHELF_NAME,
    L.FLOOR_NO,
    B.BOOK_TITLE
FROM BOOK B
RIGHT JOIN LOCATION L
    ON B.LOCATION_ID = L.LOCATION_ID
WHERE B.BOOK_ID IS NULL;
```

	LOCATION_ID	SHELF_NO	SHELF_NAME	FLOOR_NO	BOOK_TITLE
▶	L008	402	Mathematics Section	4	NULL

```
-- Audit books and categories – FULL OUTER JOIN alternative
-- Find all books and categories, including books without matching
categories and categories without books.
-- MySQL does not directly support FULL OUTER JOIN, so use LEFT JOIN and
RIGHT JOIN with UNION.
SELECT
    B.BOOK_ID,
    B.BOOK_TITLE,
```

```
    C.CATEGORY_ID,
    C.CATEGORY_NAME
FROM BOOK B
LEFT JOIN CATEGORY C
    ON B.CATEGORY_ID = C.CATEGORY_ID

UNION

SELECT
    B.BOOK_ID,
    B.BOOK_TITLE,
    C.CATEGORY_ID,
    C.CATEGORY_NAME
FROM BOOK B
RIGHT JOIN CATEGORY C
    ON B.CATEGORY_ID = C.CATEGORY_ID;
```

BOOK_ID	BOOK_TITLE	CATEGORY_ID	CATEGORY_NAME
B001	1984	C001	Fiction
B002	Harry Potter and the Philosophers Stone	C001	Fiction
B003	A Brief History of Time	C002	Science
B004	Sapiens	C003	History
B005	Wings of Fire	C005	Biography
B006	Rich Dad Poor Dad	C006	Business
B007	The Guide	C007	Literature
B008	Introduction to Algorithms	C004	Technology
B009	Clean Code	C004	Technology
B010	The C Programming Language	C004	Technology
B011	Great Expectations	C007	Literature
B012	The Alchemist	C001	Fiction
NULL	NULL	C008	Mathematics

```
-- SELF JOIN
-- Compare staff members with different designations.
-- Help management review staff role diversity.
SELECT
    S1.ISSUED_BY_ID AS STAFF_1_ID,
    S1.STAFF_NAME AS STAFF_1_NAME,
    S1.STAFF_DESIGNATION AS STAFF_1_DESIGNATION,
    S2.ISSUED_BY_ID AS STAFF_2_ID,
    S2.STAFF_NAME AS STAFF_2_NAME,
    S2.STAFF_DESIGNATION AS STAFF_2_DESIGNATION
FROM LIBRARY_STAFF S1
INNER JOIN LIBRARY_STAFF S2
    ON S1.ISSUED_BY_ID < S2.ISSUED_BY_ID
WHERE S1.STAFF_DESIGNATION <> S2.STAFF_DESIGNATION
LIMIT 10;
```

	STAFF_1_ID	STAFF_1_NAME	STAFF_1_DESIGNATION	STAFF_2_ID	STAFF_2_NAME	STAFF_2_DESIGNATION
▶	ST01	Rajesh Kumar	Chief Librarian	ST02	Anita Sharma	Librarian
	ST01	Rajesh Kumar	Chief Librarian	ST03	Vijay Patil	Assistant Librarian
	ST01	Rajesh Kumar	Chief Librarian	ST04	Meena Joshi	Library Assistant
	ST02	Anita Sharma	Librarian	ST03	Vijay Patil	Assistant Librarian
	ST02	Anita Sharma	Librarian	ST04	Meena Joshi	Library Assistant
	ST03	Vijay Patil	Assistant Librarian	ST04	Meena Joshi	Library Assistant

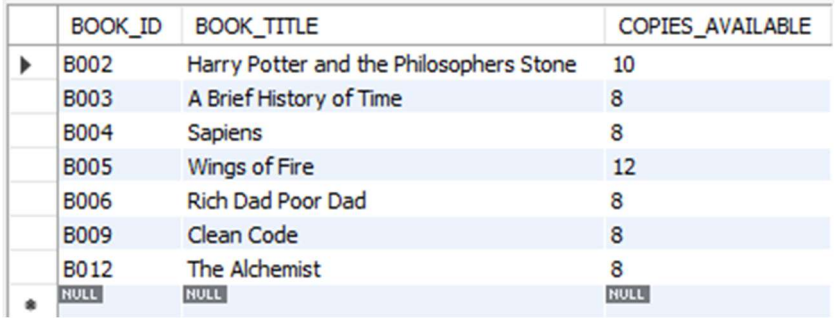
```
-- generate all possible book–staff combinations for a hypothetical
assignment plan.
SELECT
    B.BOOK_TITLE,
    S.STAFF_NAME
FROM BOOK B
CROSS JOIN LIBRARY_STAFF S;
```

	BOOK_TITLE	STAFF_NAME
▶	1984	Meena Joshi
	1984	Vijay Patil
	1984	Anita Sharma
	1984	Rajesh Kumar
	Harry Potter and the Philosophers Stone	Meena Joshi
	Harry Potter and the Philosophers Stone	Vijay Patil
	Harry Potter and the Philosophers Stone	Anita Sharma
	Harry Potter and the Philosophers Stone	Rajesh Kumar
	A Brief History of Time	Meena Joshi
	A Brief History of Time	Vijay Patil
	A Brief History of Time	Anita Sharma
	A Brief History of Time	Rajesh Kumar
	Sapiens	Meena Joshi
	Sapiens	Vijay Patil
	Sapiens	Anita Sharma
	Sapiens	Rajesh Kumar
	Wings of Fire	Meena Joshi

6. SUB QUERIES

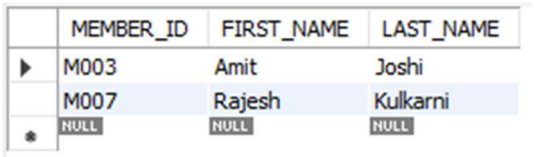
```
-- Identify books with above-average available copies
-- Identify books with higher-than-average stock availability to understand
inventory distribution.
SELECT
    BOOK_ID,
    BOOK_TITLE,
    COPIES_AVAILABLE
FROM BOOK
WHERE COPIES_AVAILABLE > (
```

```
    SELECT AVG(COPIES_AVAILABLE)
    FROM BOOK
);
```



	BOOK_ID	BOOK_TITLE	COPIES_AVAILABLE
▶	B002	Harry Potter and the Philosophers Stone	10
	B003	A Brief History of Time	8
	B004	Sapiens	8
	B005	Wings of Fire	12
	B006	Rich Dad Poor Dad	8
	B009	Clean Code	8
	B012	The Alchemist	8
*	NULL	NULL	NULL

```
-- Identify members with above-average total fines
-- Identify members whose accumulated fines exceed the average
-- individual fine amount recorded in the library.
SELECT
    M.MEMBER_ID,
    M.FIRST_NAME,
    M.LAST_NAME
FROM MEMBERS M
WHERE M.MEMBER_ID IN (
    SELECT MEMBER_ID
    FROM FINE_DUE
    GROUP BY MEMBER_ID
    HAVING SUM(FINE_TOTAL) > (
        SELECT AVG(FINE_TOTAL)
        FROM FINE_DUE
    )
);
```



	MEMBER_ID	FIRST_NAME	LAST_NAME
▶	M003	Amit	Joshi
	M007	Rajesh	Kulkarni
*	NULL	NULL	NULL

```
-- Identify members who borrow more than the average
-- Find members whose number of borrowing records exceeds the
-- average number of issue records per borrowing member.
SELECT
    M.MEMBER_ID,
    M.FIRST_NAME,
    M.LAST_NAME
FROM MEMBERS M
WHERE M.MEMBER_ID IN (
```

```
    SELECT MEMBER_ID
    FROM BOOK_ISSUE
    GROUP BY MEMBER_ID
    HAVING COUNT(*) > (
        SELECT AVG(ISSUE_COUNT)
        FROM (
            SELECT COUNT(*) AS ISSUE_COUNT
            FROM BOOK_ISSUE
            GROUP BY MEMBER_ID
        ) AS TEMP
    )
);
```

MEMBER_ID	FIRST_NAME	LAST_NAME
M001	Rahul	Sharma
M002	Priya	Patel
M003	Amit	Joshi
M004	Sneha	Desai
NULL	NULL	NULL

```
-- Identify books with the highest available copies in each category
-- Find the best-stocked book or books within each category.
SELECT
    B.BOOK_ID,
    B.BOOK_TITLE,
    B.CATEGORY_ID,
    B.COPIES_AVAILABLE
FROM BOOK B
WHERE B.COPIES_AVAILABLE = (
    SELECT MAX(B2.COPIES_AVAILABLE)
    FROM BOOK B2
    WHERE B2.CATEGORY_ID = B.CATEGORY_ID
);
```

BOOK_ID	BOOK_TITLE	CATEGORY_ID	COPIES_AVAILABLE
B002	Harry Potter and the Philosophers Stone	C001	10
B003	A Brief History of Time	C002	8
B004	Sapiens	C003	8
B005	Wings of Fire	C005	12
B006	Rich Dad Poor Dad	C006	8
B007	The Guide	C007	5
B009	Clean Code	C004	8
NULL	NULL	NULL	NULL

```
-- Identify books that have never been issued
```

```
-- Find books with no borrowing records to identify potentially underused
items.
SELECT
    B.BOOK_ID,
    B.BOOK_TITLE,
    B.CATEGORY_ID,
    B.COPIES_AVAILABLE
FROM BOOK B
WHERE NOT EXISTS (
    SELECT 1
    FROM BOOK_ISSUE I
    WHERE I.BOOK_ID = B.BOOK_ID
);
```

BOOK_ID	BOOK_TITLE	CATEGORY_ID	COPIES_AVAILABLE
B007	The Guide	C007	5
B011	Great Expectations	C007	4
NULL	NULL	NULL	NULL

7. SET OPERATORS

UNION

```
-- Generate a unified report of members who have borrowed books or made
fine payments
-- Business objective: Identify all members who have interacted financially
or operationally
-- with the library, whether by borrowing a book or making a fine payment.
SELECT MEMBER_ID,
    'Book Borrowing' AS ACTIVITY_TYPE
FROM BOOK_ISSUE

UNION

SELECT MEMBER_ID,
    'Fine Payment' AS ACTIVITY_TYPE
FROM FINE_PAYMENT;
```

	MEMBER_ID	ACTIVITY_TYPE
▶	M001	Book Borrowing
	M002	Book Borrowing
	M003	Book Borrowing
	M004	Book Borrowing
	M007	Book Borrowing
	M008	Book Borrowing
	M001	Fine Payment
	M003	Fine Payment
	M004	Fine Payment
	M007	Fine Payment

UNION ALL

Create one list containing all member first names and all staff names.

```
SELECT FIRST_NAME
FROM MEMBERS
UNION ALL
SELECT STAFF_NAME
FROM LIBRARY_STAFF;
```

	FIRST_NAME
▶	Rahul
	Priya
	Amit
	Sneha
	Vikas
	Neha
	Rajesh
	Anjali
	Rohan
	Rajesh Kumar
	Anita Sharma
	Vijay Patil
	Meena Joshi

8. CTE COMMON TABLE EXPRESSION

```
-- Category-wise inventory analysis
-- Compare total stock and available stock across book categories to
support inventory planning.
WITH CATEGORY_SUMMARY AS (
    SELECT
        CATEGORY_ID,
        SUM(COPIES_TOTAL) AS TOTAL_COPIES,
        ROUND(AVG(COPIES_AVAILABLE), 2) AS AVG_AVAILABLE_COPIES,
```

```
        MIN(COPIES_AVAILABLE) AS MIN_AVAILABLE_COPIES,
        MAX(COPIES_AVAILABLE) AS MAX_AVAILABLE_COPIES
    FROM BOOK
    GROUP BY CATEGORY_ID
)
SELECT *
FROM CATEGORY_SUMMARY
ORDER BY TOTAL_COPIES DESC;
```

CATEGORY_ID	TOTAL_COPIES	AVG_AVAILABLE_COPIES	MIN_AVAILABLE_COPIES	MAX_AVAILABLE_COPIES
C001	32	8.33	7	10
C004	26	6.67	6	8
C005	15	12.00	12	12
C007	13	4.50	4	5
C002	10	8.00	8	8
C003	10	8.00	8	8
C006	10	8.00	8	8

```
-- Identify members with above-average total fines
-- Identify members whose accumulated fines exceed the average total fines
per member.
WITH TOTAL_FINE_PER_MEMBER AS (
    SELECT
        MEMBER_ID,
        SUM(FINE_TOTAL) AS TOTAL_FINE
    FROM FINE_DUE
    GROUP BY MEMBER_ID
),
AVERAGE_FINE AS (
    SELECT AVG(TOTAL_FINE) AS AVG_TOTAL_FINE
    FROM TOTAL_FINE_PER_MEMBER
)
SELECT
    T.MEMBER_ID,
    T.TOTAL_FINE
FROM TOTAL_FINE_PER_MEMBER T
CROSS JOIN AVERAGE_FINE A
WHERE T.TOTAL_FINE > A.AVG_TOTAL_FINE
ORDER BY T.TOTAL_FINE DESC;
```

MEMBER_ID	TOTAL_FINE
M003	150.00

```
-- Identify the top 3 most borrowed books
-- Determine which books have the highest borrowing frequency to
```

```sql
-- understand demand and plan inventory.
WITH BOOK_ISSUE_COUNT AS (
    SELECT
        BOOK_ID,
        COUNT(*) AS TOTAL_ISSUES
    FROM BOOK_ISSUE
    GROUP BY BOOK_ID
)
SELECT
    B.BOOK_ID,
    B.BOOK_TITLE,
    C.TOTAL_ISSUES
FROM BOOK_ISSUE_COUNT C
INNER JOIN BOOK B
    ON C.BOOK_ID = B.BOOK_ID
ORDER BY C.TOTAL_ISSUES DESC
LIMIT 3;
```

BOOK_ID	BOOK_TITLE	TOTAL_ISSUES
B001	1984	1
B002	Harry Potter and the Philosophers Stone	1
B003	A Brief History of Time	1

```sql
-- Identify members who borrow more than average
-- Find members whose borrowing activity exceeds the average number of
issue records per member.
WITH COUNT_ISSUES AS (
    SELECT
        MEMBER_ID,
        COUNT(*) AS ISSUE_COUNT
    FROM BOOK_ISSUE
    GROUP BY MEMBER_ID
),
AVERAGE_ISSUES AS (
    SELECT AVG(ISSUE_COUNT) AS AVG_ISSUE_COUNT
    FROM COUNT_ISSUES
)
SELECT
    M.MEMBER_ID,
    M.FIRST_NAME,
    M.LAST_NAME,
    C.ISSUE_COUNT
FROM COUNT_ISSUES C
INNER JOIN MEMBERS M
    ON C.MEMBER_ID = M.MEMBER_ID
CROSS JOIN AVERAGE_ISSUES A
WHERE C.ISSUE_COUNT > A.AVG_ISSUE_COUNT
```

```
ORDER BY C.ISSUE_COUNT DESC;
```

MEMBER_ID	FIRST_NAME	LAST_NAME	ISSUE_COUNT
M001	Rahul	Sharma	2
M002	Priya	Patel	2
M003	Amit	Joshi	2
M004	Sneha	Desai	2

```
-- Rank books by available copies
-- Rank books according to their current stock availability and identify
the top three distinct stock levels.
WITH TOP_COPIES_RANKS AS (
    SELECT
        BOOK_ID,
        BOOK_TITLE,
        COPIES_TOTAL,
        COPIES_AVAILABLE,
        DENSE_RANK() OVER (
            ORDER BY COPIES_AVAILABLE DESC
        ) AS COPIES_RANK
    FROM BOOK
)
SELECT *
FROM TOP_COPIES_RANKS
WHERE COPIES_RANK <= 3
ORDER BY COPIES_RANK, BOOK_TITLE;
```

BOOK_ID	BOOK_TITLE	COPIES_TOTAL	COPIES_AVAILABLE	COPIES_RANK
B005	Wings of Fire	15	12	1
B002	Harry Potter and the Philosophers Stone	12	10	2
B003	A Brief History of Time	10	8	3
B009	Clean Code	10	8	3
B006	Rich Dad Poor Dad	10	8	3
B004	Sapiens	10	8	3
B012	The Alchemist	10	8	3

9. WINDOW FUNCTION

```
-- Find the top 2 books in every category
-- Identify the two books with the highest available stock in each category
to support inventory planning.
SELECT *
FROM (
    SELECT
        BOOK_TITLE,
```

```
        CATEGORY_ID,
        COPIES_AVAILABLE,
        ROW_NUMBER() OVER (
            PARTITION BY CATEGORY_ID
            ORDER BY COPIES_AVAILABLE DESC
        ) AS STOCK_RANK
    FROM BOOK
) AS T
WHERE STOCK_RANK <= 2;
```

BOOK_TITLE	CATEGORY_ID	COPIES_AVAILABLE	STOCK_RANK
Harry Potter and the Philosophers Stone	C001	10	1
The Alchemist	C001	8	2
A Brief History of Time	C002	8	1
Sapiens	C003	8	1
Clean Code	C004	8	1
Introduction to Algorithms	C004	6	2
Wings of Fire	C005	12	1
Rich Dad Poor Dad	C006	8	1
The Guide	C007	5	1
Great Expectations	C007	4	2

```
-- Rank fine records from highest to lowest
-- Identify the highest fine amounts and compare fine records across the
library.
SELECT
    FINE_ID,
    MEMBER_ID,
    FINE_TOTAL,
    RANK() OVER (
        ORDER BY FINE_TOTAL DESC
    ) AS FINE_RANK
FROM FINE_DUE;
```

FINE_ID	MEMBER_ID	FINE_TOTAL	FINE_RANK
F002	M003	100.00	1
F003	M007	75.00	2
F001	M001	50.00	3
F004	M003	50.00	3
F005	M004	25.00	5

```
-- Segment members into four borrowing groups
-- Classify members into four groups according to their borrowing activity,
-- helping analyze engagement levels.
SELECT
    MEMBER_ID,
    COUNT(*) AS TOTAL_ISSUES,
```

```
    NTILE(4) OVER (
        ORDER BY COUNT(*)
    ) AS BORROWING_GROUP
FROM BOOK_ISSUE
GROUP BY MEMBER_ID;
```

MEMBER_ID	TOTAL_ISSUES	BORROWING_GROUP
M007	1	1
M008	1	1
M001	2	2
M002	2	2
M003	2	3
M004	2	4

```
-- Compare each book's stock with the previous book
-- Analyze differences in available stock between books when ordered by
availability.
SELECT
    BOOK_TITLE,
    COPIES_AVAILABLE,
    LAG(COPIES_AVAILABLE) OVER (
        ORDER BY COPIES_AVAILABLE, BOOK_ID
    ) AS PREVIOUS_BOOK_AVAILABLE,
    COPIES_AVAILABLE -
    LAG(COPIES_AVAILABLE) OVER (
        ORDER BY COPIES_AVAILABLE, BOOK_ID
    ) AS STOCK_DIFFERENCE
FROM BOOK;
```

BOOK_TITLE	COPIES_AVAILABLE	PREVIOUS_BOOK_AVAILABLE	STOCK_DIFFERENCE
Great Expectations	4	NULL	NULL
The Guide	5	4	1
Introduction to Algorithms	6	5	1
The C Programming Language	6	6	0
1984	7	6	1
A Brief History of Time	8	7	1
Sapiens	8	8	0
Rich Dad Poor Dad	8	8	0
Clean Code	8	8	0
The Alchemist	8	8	0
Harry Potter and the Philoso...	10	8	2
Wings of Fire	12	10	2

```
-- Compare book availability with its category average
```

```
-- Identify books with above-average or below-average availability relative
to other books in the same category.
SELECT
    BOOK_TITLE,
    CATEGORY_ID,
    COPIES_AVAILABLE,
    ROUND(
        AVG(COPIES_AVAILABLE) OVER (
            PARTITION BY CATEGORY_ID
        ), 2
    ) AS CATEGORY_AVERAGE,
    ROUND(
        COPIES_AVAILABLE -
        AVG(COPIES_AVAILABLE) OVER (
            PARTITION BY CATEGORY_ID
        ), 2
    ) AS STOCK_DIFFERENCE
FROM BOOK
ORDER BY CATEGORY_ID, STOCK_DIFFERENCE DESC;
```

BOOK_TITLE	CATEGORY_ID	COPIES_AVAILABLE	CATEGORY_AVERAGE	STOCK_DIFFERENCE
Harry Potter and the Philosophers Stone	C001	10	8.33	1.67
The Alchemist	C001	8	8.33	-0.33
1984	C001	7	8.33	-1.33
A Brief History of Time	C002	8	8.00	0.00
Sapiens	C003	8	8.00	0.00
Clean Code	C004	8	6.67	1.33
Introduction to Algorithms	C004	6	6.67	-0.67
The C Programming Language	C004	6	6.67	-0.67
Wings of Fire	C005	12	12.00	0.00
Rich Dad Poor Dad	C006	8	8.00	0.00
The Guide	C007	5	4.50	0.50
Great Expectations	C007	4	4.50	-0.50

10. VIEW

```
-- Book stock classification
-- Identify books that are out of stock, have low stock, or have sufficient
stock.
CREATE OR REPLACE VIEW VW_BOOK_STOCK_GRADE AS
SELECT
    BOOK_ID,
    BOOK_TITLE,
    COPIES_AVAILABLE,
    CASE
        WHEN COPIES_AVAILABLE = 0 THEN 'OUT OF STOCK'
        WHEN COPIES_AVAILABLE BETWEEN 1 AND 4 THEN 'LOW STOCK'
```

```
        ELSE 'AVAILABLE'
    END AS STOCK_STATUS
FROM BOOK;
```

```
200 19:10:36 CREATE OR REPLACE VIEW VW_BOOK_STOCK_GRADE AS SELECT  BOOK_ID,  BOOK_TITLE,  C... 0 row(s) affected
```

```
SELECT *
FROM VW_BOOK_STOCK_GRADE;
```

BOOK_ID	BOOK_TITLE	COPIES_AVAILABLE	STOCK_STATUS
B001	1984	7	AVAILABLE
B002	Harry Potter and the Philosophers Stone	10	AVAILABLE
B003	A Brief History of Time	8	AVAILABLE
B004	Sapiens	8	AVAILABLE
B005	Wings of Fire	12	AVAILABLE
B006	Rich Dad Poor Dad	8	AVAILABLE
B007	The Guide	5	AVAILABLE
B008	Introduction to Algorithms	6	AVAILABLE
B009	Clean Code	8	AVAILABLE
B010	The C Programming Language	6	AVAILABLE
B011	Great Expectations	4	LOW STOCK
B012	The Alchemist	8	AVAILABLE

```
-- Member issue count
-- Measure member engagement by identifying how many books each member has
borrowed.
CREATE OR REPLACE VIEW VW_MEMBER_ISSUE_COUNT AS
SELECT
    M.MEMBER_ID,
    CONCAT(M.FIRST_NAME, ' ', M.LAST_NAME) AS MEMBER_NAME,
    COUNT(BI.ISSUE_ID) AS ISSUE_COUNT
FROM MEMBERS M
LEFT JOIN BOOK_ISSUE BI
    ON M.MEMBER_ID = BI.MEMBER_ID
GROUP BY
    M.MEMBER_ID,
    M.FIRST_NAME,
    M.LAST_NAME;
```

```
202 19:10:36 CREATE OR REPLACE VIEW VW_MEMBER_ISSUE_COUNT AS SELECT  M.MEMBER_... 0 row(s) affected
```

```
SELECT *
FROM VW_MEMBER_ISSUE_COUNT
ORDER BY ISSUE_COUNT DESC;
```

	MEMBER_ID	MEMBER_NAME	ISSUE_COUNT
▶	M001	Rahul Sharma	2
	M002	Priya Patel	2
	M003	Amit Joshi	2
	M004	Sneha Desai	2
	M007	Rajesh Kulkarni	1
	M008	Anjali Gupta	1
	M005	Vikas Mehta	0
	M006	Neha Shah	0
	M009	Rohan Mehta	0

```
-- Category-wise book count
-- Understand the distribution of books across library categories and
identify categories with fewer titles.
CREATE OR REPLACE VIEW VW_CATEGORY_BOOK_COUNT AS
SELECT
    C.CATEGORY_ID,
    C.CATEGORY_NAME,
    COUNT(B.BOOK_ID) AS BOOK_COUNT
FROM CATEGORY C
LEFT JOIN BOOK B
    ON C.CATEGORY_ID = B.CATEGORY_ID
GROUP BY
    C.CATEGORY_ID,
    C.CATEGORY_NAME;
```




```
SELECT *
FROM VW_CATEGORY_BOOK_COUNT
ORDER BY BOOK_COUNT DESC;
```

	CATEGORY_ID	CATEGORY_NAME	BOOK_COUNT
▶	C001	Fiction	3
	C004	Technology	3
	C007	Literature	2
	C002	Science	1
	C003	History	1
	C005	Biography	1
	C006	Business	1
	C008	Mathematics	0

```
-- Fine classification
-- Categorize member fines into low, medium, and high amounts to help
prioritize fine follow-up.
```

```
CREATE OR REPLACE VIEW VW_FINE_CLASSIFICATION AS
SELECT
    FINE_ID,
    MEMBER_ID,
    FINE_TOTAL,
    CASE
        WHEN FINE_TOTAL < 50 THEN 'LOW FINE'
        WHEN FINE_TOTAL <= 100 THEN 'MEDIUM FINE'
        ELSE 'HIGH FINE'
    END AS FINE_CATEGORY
FROM FINE_DUE;
```

206 19:10:36 CREATE OR REPLACE VIEW VW_FINE_CLASSIFICATION AS SELECT FINE_ID, ME... 0 row(s) affected

```
SELECT *
FROM VW_FINE_CLASSIFICATION
ORDER BY FINE_TOTAL DESC;
```

FINE_ID	MEMBER_ID	FINE_TOTAL	FINE_CATEGORY
F002	M003	100.00	MEDIUM FINE
F003	M007	75.00	MEDIUM FINE
F001	M001	50.00	MEDIUM FINE
F004	M003	50.00	MEDIUM FINE
F005	M004	25.00	LOW FINE

```
-- Staff issue count
-- Compare the number of book issue transactions handled by each staff
member.
CREATE OR REPLACE VIEW VW_STAFF_ISSUE_COUNT AS
SELECT
    LS.ISSUED_BY_ID,
    LS.STAFF_NAME,
    LS.STAFF_DESIGNATION,
    COUNT(BI.ISSUE_ID) AS ISSUE_COUNT
FROM LIBRARY_STAFF LS
LEFT JOIN BOOK_ISSUE BI
    ON LS.ISSUED_BY_ID = BI.ISSUED_BY_ID
GROUP BY
    LS.ISSUED_BY_ID,
    LS.STAFF_NAME,
    LS.STAFF_DESIGNATION;
```

208 19:10:36 CREATE OR REPLACE VIEW VW_STAFF_ISSUE_COUNT AS SELECT LS.ISSUED_BY... 0 row(s) affected

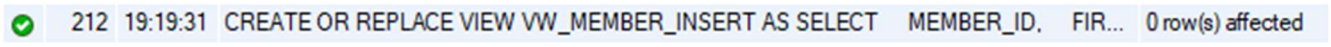
```
SELECT *
FROM VW_STAFF_ISSUE_COUNT
ORDER BY ISSUE_COUNT DESC;
```

	ISSUED_BY_ID	STAFF_NAME	STAFF_DESIGNATION	ISSUE_COUNT
▶	ST01	Rajesh Kumar	Chief Librarian	3
	ST02	Anita Sharma	Librarian	3
	ST03	Vijay Patil	Assistant Librarian	2
	ST04	Meena Joshi	Library Assistant	2

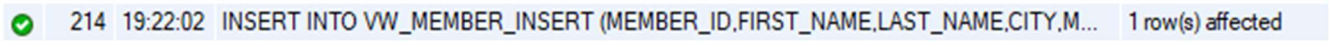
```
-- INSERT, UPDATE and DELETE through views
```

Create a simple view containing the necessary columns from MEMBERS,

```
-- then insert a new member through that view.
CREATE OR REPLACE VIEW VW_MEMBER_INSERT AS
SELECT
    MEMBER_ID,
    FIRST_NAME,
    LAST_NAME,
    CITY,
    MOBILE_NO,
    EMAIL_ID,
    DATE_OF_BIRTH,
    ACTIVE_STATUS_ID
FROM MEMBERS;
```




```
INSERT INTO VW_MEMBER_INSERT (MEMBER_ID,FIRST_NAME,LAST_NAME,CITY,
MOBILE_NO,EMAIL_ID,DATE_OF_BIRTH)
VALUES (
    'M010','Ryan','Ron','Mumbai','9876543219','ryanron@gmail.com','2026-10-
20'
);
```



```
-- Create a simple member view and practise deleting a test member through
it.
CREATE OR REPLACE VIEW VW_MEMBER_DELETE AS
SELECT
    MEMBER_ID,
    FIRST_NAME,
    LAST_NAME,
    CITY
FROM MEMBERS;
```

```
216 19:24:30 CREATE OR REPLACE VIEW VW_MEMBER_DELETE AS SELECT    MEMBER_ID,    FIR... 0 row(s) affected
```

```
DELETE FROM VW_MEMBER_DELETE
WHERE MEMBER_ID = 'M009';
```

```
217 19:24:30 DELETE FROM VW_MEMBER_DELETE WHERE MEMBER_ID = 'M009'                1 row(s) affected
```

```
SELECT *
FROM VW_MEMBER_DELETE;
```

MEMBER_ID	FIRST_NAME	LAST_NAME	CITY
M001	Rahul	Sharma	Mumbai
M002	Priya	Patel	Mumbai
M003	Amit	Joshi	Thane
M004	Sneha	Desai	Mumbai
M005	Vikas	Mehta	Navi Mumbai
M006	Neha	Shah	Mumbai
M007	Rajesh	Kulkarni	Pune
M008	Anjali	Gupta	Mumbai
M010	Ryan	Ron	Mumbai

```
-- Create a simple view containing MEMBER_ID, FIRST_NAME, and CITY. Use it
to change the city of a test member.
CREATE OR REPLACE VIEW VW_MEMBER_CITY AS
SELECT
    MEMBER_ID,
    FIRST_NAME,
    CITY
FROM MEMBERS;
```

```
219 19:25:50 CREATE OR REPLACE VIEW VW_MEMBER_CITY AS SELECT    MEMBER_ID,    FIRST... 0 row(s) affected
```

```
UPDATE VW_MEMBER_CITY
```

```
SET CITY = 'New York'
WHERE MEMBER_ID = 'M001';
```

```
220 19:25:50 UPDATE VW_MEMBER_CITY SET CITY = 'New York' WHERE MEMBER_ID = 'M001' 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
```

```
SELECT *
FROM VW_MEMBER_CITY;
```

MEMBER_ID	FIRST_NAME	CITY
M001	Rahul	New York
M002	Priya	Mumbai
M003	Amit	Thane
M004	Sneha	Mumbai
M005	Vikas	Navi Mumbai
M006	Neha	Mumbai
M007	Rajesh	Pune
M008	Anjali	Mumbai
M010	Ryan	Mumbai

11. TCL

```
-- Issue a book and update available stock
-- When a book is issued, decrease its available copies and record the
transaction only if the issue is successful.
START TRANSACTION;
```

```
223 19:28:19 START TRANSACTION 0 row(s) affected
```

```
UPDATE BOOK
SET COPIES_AVAILABLE = COPIES_AVAILABLE - 1
WHERE BOOK_ID = 101
  AND COPIES_AVAILABLE > 0;
```

```
225 19:28:44 UPDATE BOOK SET COPIES_AVAILABLE = COPIES_AVAILABLE - 1 WHERE BOOK_ID = '... 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
```

```
INSERT INTO BOOK_ISSUE
    (ISSUE_ID, BOOK_ID, MEMBER_ID, ISSUE_DATE, ISSUE_STATUS)
VALUES
    ('I011', 'B007', 'M001', CURDATE(), 'ISSUED');
```

```
229 19:31:34 INSERT INTO BOOK_ISSUE (ISSUE_ID, BOOK_ID, MEMBER_ID, ISSUE_DATE, ISSUE... 1 row(s) affected
```

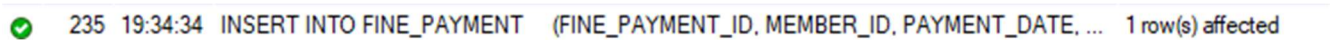
```
COMMIT;
```




```
-- Roll back an incorrect fine payment
-- If an incorrect fine payment is entered, cancel the transaction before
it is committed.
START TRANSACTION;
```




```
INSERT INTO FINE_PAYMENT
    (FINE_PAYMENT_ID, MEMBER_ID, PAYMENT_DATE, PAYMENT_AMOUNT)
VALUES
    ('FP05', 'M001', CURDATE(), 500.00);
```



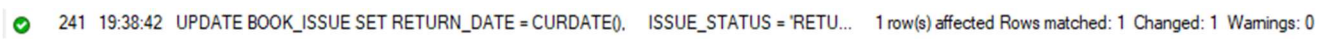

```
ROLLBACK;
```




```
-- Use a savepoint when processing a book return
-- Record a book return, then use a savepoint so the return can be retained
even
-- if a subsequent inventory adjustment must be undone.
START TRANSACTION;
```




```
UPDATE BOOK_ISSUE
SET RETURN_DATE = CURDATE(),
    ISSUE_STATUS = 'RETURNED'
WHERE ISSUE_ID = ‘I005’
  AND ISSUE_STATUS = 'ISSUED';
```



```
SAVEPOINT RETURN_RECORDED;
```

```
239  19:37:34  SAVEPOINT RETURN_RECORDED                    0 row(s) affected
```

```
UPDATE BOOK
SET COPIES_AVAILABLE = COPIES_AVAILABLE + 1
WHERE BOOK_ID = 'B001';
```

```
240  19:37:34  UPDATE BOOK SET COPIES_AVAILABLE = COPIES_AVAILABLE + 1 WHERE BOOK_ID =...  1 row(s) affected Rows matched: 1  Changed: 1  Warnings: 0
```

```
ROLLBACK TO SAVEPOINT RETURN_RECORDED;
```

```
242  19:40:06  ROLLBACK TO SAVEPOINT RETURN_RECORDED    0 row(s) affected
```

```
COMMIT;
```

```
243  19:40:09  COMMIT                                    0 row(s) affected
```

```
-- Transfer a fine amount between accounting records
-- Correct a fine payment that was mistakenly assigned to one member and
reassign it
-- to the correct member as a single transaction.
START TRANSACTION;
```

```
251  19:44:01  START TRANSACTION                         0 row(s) affected
```

```
UPDATE FINE_PAYMENT
SET MEMBER_ID = 'M003'
WHERE FINE_PAYMENT_ID = 'FP01'
  AND MEMBER_ID = 'M001';
```

```
260  19:50:42  UPDATE FINE_PAYMENT SET MEMBER_ID = 'M003' WHERE FINE_PAYMENT_ID = 'FP0...  1 row(s) affected Rows matched: 1  Changed: 1  Warnings: 0
```

```
COMMIT;
```

```
261  19:50:59  COMMIT                                                    0 row(s) affected
```

```
-- Delete a duplicate issue record safely
-- Remove an incorrectly duplicated book-issue record, but preserve it if validation reveals the deletion was wrong.
START TRANSACTION;
```

```
269  19:53:52  START TRANSACTION                                        0 row(s) affected
```

```
SAVEPOINT BEFORE_DELETE;
```

```
270  19:53:52  SAVEPOINT BEFORE_DELETE                                 0 row(s) affected
```

```
DELETE FROM FINE_DUE
WHERE ISSUE_ID = 'I010';
```

```
271  19:53:52  DELETE FROM FINE_DUE WHERE ISSUE_ID = 'I010'            1 row(s) affected
```

```
DELETE FROM BOOK_ISSUE
WHERE ISSUE_ID = 'I010';
```

```
272  19:53:52  DELETE FROM BOOK_ISSUE WHERE ISSUE_ID = 'I010'          1 row(s) affected
```

```
COMMIT;
```

```
273  19:53:52  COMMIT                                                    0 row(s) affected
```

12. STORED PROCEDURE

```
-- Book inventory analysis by category
-- Management wants to analyze total stock, available copies,
-- borrowed copies, and inventory utilization for a selected book category.
DELIMITER //

CREATE PROCEDURE SP_CATEGORY_INVENTORY_ANALYSIS(
    IN P_CATEGORY_ID VARCHAR(10)
```

```
)
BEGIN
    SELECT
        C.CATEGORY_ID,
        C.CATEGORY_NAME,
        COUNT(B.BOOK_ID) AS TOTAL_BOOK_TITLES,
        COALESCE(SUM(B.COPIES_TOTAL), 0) AS TOTAL_COPIES,
        COALESCE(SUM(B.COPIES_AVAILABLE), 0) AS AVAILABLE_COPIES,
        COALESCE(
            SUM(B.COPIES_TOTAL - B.COPIES_AVAILABLE), 0
        ) AS BORROWED_COPIES,
        ROUND(
            100.0 * SUM(B.COPIES_AVAILABLE)
            / NULLIF(SUM(B.COPIES_TOTAL), 0),
            2
        ) AS AVAILABILITY_PERCENTAGE
    FROM CATEGORY C
    LEFT JOIN BOOK B
        ON C.CATEGORY_ID = B.CATEGORY_ID
    WHERE C.CATEGORY_ID = P_CATEGORY_ID
    GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME;
END //



DELIMITER ;
```

✔	274	19:56:13	CREATE PROCEDURE SP_CATEGORY_INVENTORY_ANALYSIS(	IN P_CATEGORY_I...	0 row(s) affected

```
CALL SP_CATEGORY_INVENTORY_ANALYSIS('C004');
```

	CATEGORY_ID	CATEGORY_NAME	TOTAL_BOOK_TITLES	TOTAL_COPIES	AVAILABLE_COPIES	BORROWED_COPIES	AVAILABILITY_PERCENTAGE
▶	C004	Technology	3	26	20	6	76.92

```
-- Book borrowing analysis by date range
-- Management wants to know which books are borrowed most frequently during
a selected period.
DELIMITER //

CREATE PROCEDURE SP_BOOK_BORROWING_ANALYSIS(
    IN P_START_DATE DATE,
    IN P_END_DATE DATE
)
BEGIN
    SELECT
        B.BOOK_ID,
```

```
        B.BOOK_TITLE,
        C.CATEGORY_NAME,
        COUNT(BI.ISSUE_ID) AS TOTAL_ISSUES,
        SUM(
            CASE
                WHEN BI.ISSUE_STATUS = 'Issued' THEN 1
                ELSE 0
            END
        ) AS CURRENTLY_ISSUED_COUNT,
        SUM(
            CASE
                WHEN BI.ISSUE_STATUS = 'Returned' THEN 1
                ELSE 0
            END
        ) AS RETURNED_COUNT
    FROM BOOK B
    INNER JOIN CATEGORY C
        ON B.CATEGORY_ID = C.CATEGORY_ID
    INNER JOIN BOOK_ISSUE BI
        ON B.BOOK_ID = BI.BOOK_ID
    WHERE BI.ISSUE_DATE BETWEEN P_START_DATE AND P_END_DATE
    GROUP BY B.BOOK_ID, B.BOOK_TITLE, C.CATEGORY_NAME
    ORDER BY TOTAL_ISSUES DESC;
END //

DELIMITER ;
```

✓	276	19:57:59	CREATE PROCEDURE SP_BOOK_BORROWING_ANALYSIS(	IN P_START_DATE DAT...	0 row(s) affected

```
CALL SP_BOOK_BORROWING_ANALYSIS(
    '2026-09-01',
    '2026-09-30'
);
```

BOOK_ID	BOOK_TITLE	CATEGORY_NAME	TOTAL_ISSUES	CURRENTLY_ISSUED_COUNT	RETURNED_COUNT
B001	1984	Fiction	1	0	1
B003	A Brief History of Time	Science	1	0	1
B005	Wings of Fire	Biography	1	0	1
B006	Rich Dad Poor Dad	Business	1	0	1
B008	Introduction to Algorithms	Technology	1	0	1
B002	Harry Potter and the Philosophers Stone	Fiction	1	0	1
B009	Clean Code	Technology	1	1	0
B004	Sapiens	History	1	0	1
B010	The C Programming Language	Technology	1	1	0

```
-- Member borrowing and membership status analysis
-- Management wants to evaluate borrowing activity for a selected
membership status, such as Active or Expired.
```

```
DELIMITER //

CREATE PROCEDURE SP_MEMBER_ACTIVITY_ANALYSIS(
    IN P_ACCOUNT_STATUS VARCHAR(20)
)
BEGIN
    SELECT
        M.MEMBER_ID,
        CONCAT(M.FIRST_NAME, ' ', M.LAST_NAME) AS MEMBER_NAME,
        M.CITY,
        MS.ACCOUNT_TYPE,
        MS.ACCOUNT_STATUS,
        COUNT(BI.ISSUE_ID) AS TOTAL_BOOKS_ISSUED,
        SUM(
            CASE
                WHEN BI.ISSUE_STATUS = 'Issued' THEN 1
                ELSE 0
            END
        ) AS CURRENTLY_BORROWED,
        MAX(BI.ISSUE_DATE) AS LAST_ISSUE_DATE
    FROM MEMBERS M
    INNER JOIN MEMBER_STATUS MS
        ON M.ACTIVE_STATUS_ID = MS.ACTIVE_STATUS_ID
    LEFT JOIN BOOK_ISSUE BI
        ON M.MEMBER_ID = BI.MEMBER_ID
    WHERE MS.ACCOUNT_STATUS = P_ACCOUNT_STATUS
    GROUP BY
        M.MEMBER_ID,
        M.FIRST_NAME,
        M.LAST_NAME,
        M.CITY,
        MS.ACCOUNT_TYPE,
        MS.ACCOUNT_STATUS
    ORDER BY TOTAL_BOOKS_ISSUED DESC;
END //



DELIMITER ;
```




```
CALL SP_MEMBER_ACTIVITY_ANALYSIS('Active');
```

MEMBER_ID	MEMBER_NAME	CITY	ACCOUNT_TYPE	ACCOUNT_STATUS	TOTAL_BOOKS_ISSUED	CURRENTLY_BORROWED	LAST_ISSUE_DATE
M001	Rahul Sharma	New York	Student	Active	3	1	2026-10-18
M002	Priya Patel	Mumbai	Student	Active	2	1	2026-09-12
M003	Amit Joshi	Thane	Faculty	Active	2	0	2026-09-11
M004	Sneha Desai	Mumbai	General	Active	1	0	2026-09-07
M007	Rajesh Kulkarni	Pune	Faculty	Active	1	0	2026-09-09
M008	Anjali Gupta	Mumbai	Student	Active	1	1	2026-09-10

```
-- Fine due versus fine payment analysis
-- Management wants to analyze total fines charged, total payments
received,
-- and outstanding fine balances for a selected member.
DELIMITER //

CREATE PROCEDURE SP_MEMBER_FINE_ANALYSIS(
    IN P_MEMBER_ID VARCHAR(10)
)
BEGIN
    SELECT
        M.MEMBER_ID,
        CONCAT(M.FIRST_NAME, ' ', M.LAST_NAME) AS MEMBER_NAME,
        COALESCE(FD.TOTAL_FINE_DUE, 0) AS TOTAL_FINE_DUE,
        COALESCE(FP.TOTAL_FINE_PAID, 0) AS TOTAL_FINE_PAID,
        COALESCE(FD.TOTAL_FINE_DUE, 0)
          - COALESCE(FP.TOTAL_FINE_PAID, 0)
            AS OUTSTANDING_FINE
    FROM MEMBERS M
    LEFT JOIN (
        SELECT
            MEMBER_ID,
            SUM(FINE_TOTAL) AS TOTAL_FINE_DUE
        FROM FINE_DUE
        GROUP BY MEMBER_ID
    ) FD
        ON M.MEMBER_ID = FD.MEMBER_ID
    LEFT JOIN (
        SELECT
            MEMBER_ID,
            SUM(PAYMENT_AMOUNT) AS TOTAL_FINE_PAID
        FROM FINE_PAYMENT
        GROUP BY MEMBER_ID
    ) FP
        ON M.MEMBER_ID = FP.MEMBER_ID
    WHERE M.MEMBER_ID = P_MEMBER_ID;
END //

DELIMITER ;
```

280	19:57:59	CREATE PROCEDURE SP_MEMBER_FINE_ANALYSIS(IN P_MEMBER_ID VARCHAR(...	0 row(s) affected

```
CALL SP_MEMBER_FINE_ANALYSIS('M003');
```

MEMBER_ID	MEMBER_NAME	TOTAL_FINE_DUE	TOTAL_FINE_PAID	OUTSTANDING_FINE
M003	Amit Joshi	150.00	150.00	0.00

```
-- Library staff performance analysis
-- Management wants to compare staff members based on the number of books
they issue during a selected period.
DELIMITER //

CREATE PROCEDURE SP_STAFF_PERFORMANCE_ANALYSIS(
    IN P_START_DATE DATE,
    IN P_END_DATE DATE
)
BEGIN
    SELECT
        LS.ISSUED_BY_ID,
        LS.STAFF_NAME,
        LS.STAFF_DESIGNATION,
        COUNT(BI.ISSUE_ID) AS TOTAL_ISSUES_HANDLED,
        SUM(
            CASE
                WHEN BI.ISSUE_STATUS = 'Issued' THEN 1
                ELSE 0
            END
        ) AS CURRENTLY_ISSUED_COUNT,
        SUM(
            CASE
                WHEN BI.ISSUE_STATUS = 'Returned' THEN 1
                ELSE 0
            END
        ) AS RETURNED_COUNT
    FROM LIBRARY_STAFF LS
    LEFT JOIN BOOK_ISSUE BI
        ON LS.ISSUED_BY_ID = BI.ISSUED_BY_ID
        AND BI.ISSUE_DATE BETWEEN P_START_DATE AND P_END_DATE
    GROUP BY
        LS.ISSUED_BY_ID,
        LS.STAFF_NAME,
        LS.STAFF_DESIGNATION
    ORDER BY TOTAL_ISSUES_HANDLED DESC;
END //

DELIMITER ;
```

✓	282	19:57:59	CREATE PROCEDURE SP_STAFF_PERFORMANCE_ANALYSIS(IN P_START_DATE D...	0 row(s) affected

```
CALL SP_STAFF_PERFORMANCE_ANALYSIS(
    '2026-09-01',
    '2026-09-30'
);
```

	ISSUED_BY_ID	STAFF_NAME	STAFF_DESIGNATION	TOTAL_ISSUES_HANDLED	CURRENTLY_ISSUED_COUNT	RETURNED_COUNT
▶	ST01	Rajesh Kumar	Chief Librarian	3	1	2
	ST02	Anita Sharma	Librarian	3	1	2
	ST03	Vijay Patil	Assistant Librarian	2	0	2
	ST04	Meena Joshi	Library Assistant	1	0	1

13. TRIGGER

```
-- Automatically update book inventory when a book is issued
-- Whenever a new book issue is recorded, automatically reduce the
available book copies.
DELIMITER //

CREATE TRIGGER TRG_AFTER_BOOK_ISSUE
AFTER INSERT ON BOOK_ISSUE
FOR EACH ROW
BEGIN
    IF NEW.ISSUE_STATUS = 'Issued' THEN
        UPDATE BOOK
        SET COPIES_AVAILABLE = COPIES_AVAILABLE - 1
        WHERE BOOK_ID = NEW.BOOK_ID
          AND COPIES_AVAILABLE > 0;
    END IF;
END //

DELIMITER ;
```

✓	284	20:59:34	CREATE TRIGGER TRG_AFTER_BOOK_ISSUE AFTER INSERT ON BOOK_ISSUE FOR E...	0 row(s) affected

```
-- Check the book's available copies before issuing
SELECT BOOK_ID, BOOK_TITLE, COPIES_AVAILABLE
FROM BOOK
WHERE BOOK_ID = 'B007';
```

	BOOK_ID	BOOK_TITLE	COPIES_AVAILABLE
▶	B007	The Guide	4
*	NULL	NULL	NULL

```
INSERT INTO BOOK_ISSUE
    (ISSUE_ID, BOOK_ID, MEMBER_ID, ISSUE_DATE, ISSUE_STATUS)
VALUES
    ('I099', 'B007', 'M001', CURDATE(), 'Issued');
```

✓	286	21:02:13	INSERT INTO BOOK_ISSUE	(ISSUE_ID, BOOK_ID, MEMBER_ID, ISSUE_DATE, ISSUE...	1 row(s) affected

```
SELECT BOOK_ID, BOOK_TITLE, COPIES_AVAILABLE
FROM BOOK
WHERE BOOK_ID = 'B007';
```

	BOOK_ID	BOOK_TITLE	COPIES_AVAILABLE
▶	B007	The Guide	3
*	NULL	NULL	NULL

```
-- Automatically restore book inventory when a book is returned
-- When a book's issue status changes from Issued to Returned, increase the
available copies by one
DELIMITER //

CREATE TRIGGER TRG_AFTER_BOOK_RETURN
AFTER UPDATE ON BOOK_ISSUE
FOR EACH ROW
BEGIN
    IF OLD.ISSUE_STATUS = 'Issued'
       AND NEW.ISSUE_STATUS = 'Returned' THEN

        UPDATE BOOK
        SET COPIES_AVAILABLE = COPIES_AVAILABLE + 1
        WHERE BOOK_ID = NEW.BOOK_ID;

    END IF;
END //

DELIMITER ;
```

✓	288	21:03:59	CREATE TRIGGER TRG_AFTER_BOOK_RETURN AFTER UPDATE ON BOOK_ISSUE FO...	0 row(s) affected

```
-- Check the issue record
SELECT ISSUE_ID, BOOK_ID, ISSUE_STATUS
FROM BOOK_ISSUE
WHERE ISSUE_ID = 'I099';
```

	ISSUE_ID	BOOK_ID	ISSUE_STATUS
▶	I099	B007	Issued
*	NULL	NULL	NULL

```
-- Mark the book as returned
UPDATE BOOK_ISSUE
SET ISSUE_STATUS = 'Returned'
WHERE ISSUE_ID = 'I099'
  AND ISSUE_STATUS = 'Issued';
```

290 21:05:36 UPDATE BOOK_ISSUE SET ISSUE_STATUS = 'Returned' WHERE ISSUE_ID = 'I099' AN... 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

```
-- Verify the updated status and inventory
SELECT ISSUE_ID, BOOK_ID, ISSUE_STATUS
FROM BOOK_ISSUE
WHERE ISSUE_ID = 'I099';
```

ISSUE_ID	BOOK_ID	ISSUE_STATUS
I099	B007	Returned
NULL	NULL	NULL

```
SELECT BOOK_ID, BOOK_TITLE, COPIES_AVAILABLE
FROM BOOK
WHERE BOOK_ID = 'B007';
```

BOOK_ID	BOOK_TITLE	COPIES_AVAILABLE
B007	The Guide	4
NULL	NULL	NULL

```
-- Maintain member-wise fine payment statistics
-- Whenever a fine payment is recorded, update a separate summary table
-- containing the member's total payments and payment count.

CREATE TABLE MEMBER_PAYMENT_STATS (
    MEMBER_ID VARCHAR(10) PRIMARY KEY,
    TOTAL_PAYMENTS INT DEFAULT 0,
    TOTAL_AMOUNT_PAID DECIMAL(10,2) DEFAULT 0.00,
    FOREIGN KEY (MEMBER_ID) REFERENCES MEMBERS(MEMBER_ID)
);
```

293 21:07:46 CREATE TABLE MEMBER_PAYMENT_STATS (MEMBER_ID VARCHAR(10) PRIMARY ... 0 row(s) affected

```
DELIMITER //

CREATE TRIGGER TRG_AFTER_FINE_PAYMENT
AFTER INSERT ON FINE_PAYMENT
FOR EACH ROW
```

```
BEGIN
    INSERT INTO MEMBER_PAYMENT_STATS
        (MEMBER_ID, TOTAL_PAYMENTS, TOTAL_AMOUNT_PAID)
    VALUES
        (NEW.MEMBER_ID, 1, NEW.PAYMENT_AMOUNT)
    ON DUPLICATE KEY UPDATE
        TOTAL_PAYMENTS = TOTAL_PAYMENTS + 1,
        TOTAL_AMOUNT_PAID =
            TOTAL_AMOUNT_PAID + NEW.PAYMENT_AMOUNT;
END //

DELIMITER ;
```

294 21:08:16 CREATE TRIGGER TRG_AFTER_FINE_PAYMENT AFTER INSERT ON FINE_PAYMENT ... 0 row(s) affected

```
-- check the summary table:
SELECT *
FROM MEMBER_PAYMENT_STATS
ORDER BY MEMBER_ID;
```

	MEMBER_ID	TOTAL_PAYMENTS	TOTAL_AMOUNT_PAID
*	NULL	NULL	NULL

```
-- Check the member's existing payment records:
SELECT
    MEMBER_ID,
    COUNT(*) AS PAYMENT_COUNT,
    SUM(PAYMENT_AMOUNT) AS TOTAL_AMOUNT_PAID
FROM FINE_PAYMENT
WHERE MEMBER_ID = 'M001'
GROUP BY MEMBER_ID;
```

	MEMBER_ID	PAYMENT_COUNT	TOTAL_AMOUNT_PAID

```
-- Insert a test payment using a unique payment ID:
INSERT INTO FINE_PAYMENT
    (FINE_PAYMENT_ID, MEMBER_ID, PAYMENT_DATE, PAYMENT_AMOUNT)
VALUES
    ('FP99', 'M001', CURDATE(), 100.00);
```

298 21:12:10 INSERT INTO FINE_PAYMENT (FINE_PAYMENT_ID, MEMBER_ID, PAYMENT_DATE, ... 1 row(s) affected

```
-- Check the updated statistics:
SELECT *
FROM MEMBER_PAYMENT_STATS
WHERE MEMBER_ID = 'M001';
```

MEMBER_ID	TOTAL_PAYMENTS	TOTAL_AMOUNT_PAID
M001	1	100.00
NULL	NULL	NULL

```
-- Prevent issuing books to expired or inactive members
-- Before inserting a book issue, verify that the member's account
-- is active and its membership end date has not passed.
DELIMITER //

CREATE TRIGGER TRG_VALIDATE_MEMBER_BEFORE_ISSUE
BEFORE INSERT ON BOOK_ISSUE
FOR EACH ROW
BEGIN
    DECLARE V_STATUS VARCHAR(20);
    DECLARE V_END_DATE DATE;

    SELECT MS.ACCOUNT_STATUS, MS.MEMBERSHIP_END_DATE
    INTO V_STATUS, V_END_DATE
    FROM MEMBERS M
    INNER JOIN MEMBER_STATUS MS
        ON M.ACTIVE_STATUS_ID = MS.ACTIVE_STATUS_ID
    WHERE M.MEMBER_ID = NEW.MEMBER_ID;

    IF V_STATUS <> 'Active'
       OR V_END_DATE < CURDATE() THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
            'Book issue denied: membership is inactive or expired';
    END IF;
END //

DELIMITER ;
```

✓	300	21:14:11	CREATE TRIGGER TRG_VALIDATE_MEMBER_BEFORE_ISSUE BEFORE INSERT ON B...	0 row(s) affected

```
-- inspect the member and membership status:
SELECT
    M.MEMBER_ID,
    M.FIRST_NAME,
    MS.ACCOUNT_STATUS,
    MS.MEMBERSHIP_END_DATE
FROM MEMBERS M
INNER JOIN MEMBER_STATUS MS
```

```
    ON M.ACTIVE_STATUS_ID = MS.ACTIVE_STATUS_ID
WHERE M.MEMBER_ID = 'M001';
```

	MEMBER_ID	FIRST_NAME	ACCOUNT_STATUS	MEMBERSHIP_END_DATE
▶	M001	Rahul	Active	2027-09-27

```
-- test a valid issue with an unused issue ID:
INSERT INTO BOOK_ISSUE
    (ISSUE_ID, BOOK_ID, MEMBER_ID, ISSUE_DATE, ISSUE_STATUS)
VALUES
    ('I100', 'B007', 'M001', CURDATE(), 'Issued');
```

302 21:16:47 INSERT INTO BOOK_ISSUE (ISSUE_ID, BOOK_ID, MEMBER_ID, ISSUE_DATE, ISSUE... 1 row(s) affected

```
-- test rejection
INSERT INTO BOOK_ISSUE
    (ISSUE_ID, BOOK_ID, MEMBER_ID, ISSUE_DATE, ISSUE_STATUS)
VALUES
    ('I101', 'B007', 'M006', CURDATE(), 'Issued');
```

305 21:22:02 INSERT INTO BOOK_ISSUE (ISSUE_ID, BOOK_ID, MEMBER_ID, ISSUE_DATE, ISSUE... Error Code: 1644. Book issue denied: membership is inactive or expired

```
-- Audit deleted book issue records
-- If a book issue record is deleted, preserve its key details and the
-- deletion timestamp for accountability and reporting.

CREATE TABLE BOOK_ISSUE_AUDIT (
    AUDIT_ID INT AUTO_INCREMENT PRIMARY KEY,
    ISSUE_ID VARCHAR(10),
    BOOK_ID VARCHAR(10),
    MEMBER_ID VARCHAR(10),
    ISSUE_DATE DATE,
    RETURN_DATE DATE,
    ISSUE_STATUS VARCHAR(20),
    DELETED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

306 21:23:25 CREATE TABLE BOOK_ISSUE_AUDIT (AUDIT_ID INT AUTO_INCREMENT PRIMARY ... 0 row(s) affected

```
DELIMITER //

CREATE TRIGGER TRG_AFTER_BOOK_ISSUE_DELETE
AFTER DELETE ON BOOK_ISSUE
FOR EACH ROW
```

```
BEGIN
    INSERT INTO BOOK_ISSUE_AUDIT (
        ISSUE_ID,
        BOOK_ID,
        MEMBER_ID,
        ISSUE_DATE,
        RETURN_DATE,
        ISSUE_STATUS
    )
    VALUES (
        OLD.ISSUE_ID,
        OLD.BOOK_ID,
        OLD.MEMBER_ID,
        OLD.ISSUE_DATE,
        OLD.RETURN_DATE,
        OLD.ISSUE_STATUS
    );
END //

DELIMITER ;
```

307 21:23:54 CREATE TRIGGER TRG_AFTER_BOOK_ISSUE_DELETE AFTER DELETE ON BOOK_ISS... 0 row(s) affected

```
-- inspect the issue record:
SELECT *
FROM BOOK_ISSUE
WHERE ISSUE_ID = 'I099';
```

ISSUE_ID	BOOK_ID	MEMBER_ID	ISSUE_DATE	RETURN_DATE	ISSUE_STATUS	ISSUED_BY_ID
I099	B007	M001	2026-10-18	NULL	Returned	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
-- Check whether any fine records reference it:
SELECT *
FROM FINE_DUE
WHERE ISSUE_ID = 'I099';
```

FINE_ID	MEMBER_ID	ISSUE_ID	FINE_DATE	FINE_TOTAL
NULL	NULL	NULL	NULL	NULL

```
-- test record, run:
DELETE FROM BOOK_ISSUE
WHERE ISSUE_ID = 'I099';
```

310 21:29:39 DELETE FROM BOOK_ISSUE WHERE ISSUE_ID = 'I099' 1 row(s) affected

```
-- Now check the audit table:
SELECT *
FROM BOOK_ISSUE_AUDIT
ORDER BY AUDIT_ID DESC
LIMIT 10;
```

AUDIT_ID	ISSUE_ID	BOOK_ID	MEMBER_ID	ISSUE_DATE	RETURN_DATE	ISSUE_STATUS	DELETED_AT
1	I099	B007	M001	2026-10-18	NULL	Returned	2026-10-18 21:29:39
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

14. KPI ANALYSIS

```
-- Book Inventory Availability KPI
-- How many copies are available, how many are borrowed, and what
percentage of inventory is available?
SELECT
    SUM(COPIES_TOTAL) AS TOTAL_COPIES,
    SUM(COPIES_AVAILABLE) AS AVAILABLE_COPIES,
    SUM(COPIES_TOTAL - COPIES_AVAILABLE) AS BORROWED_COPIES,
    ROUND(
        SUM(COPIES_AVAILABLE) * 100.0
        / NULLIF(SUM(COPIES_TOTAL), 0),
        2
    ) AS AVAILABILITY_PERCENTAGE,
    ROUND(
        SUM(COPIES_TOTAL - COPIES_AVAILABLE) * 100.0
        / NULLIF(SUM(COPIES_TOTAL), 0),
        2
    ) AS UTILIZATION_PERCENTAGE
FROM BOOK;
```

TOTAL_COPIES	AVAILABLE_COPIES	BORROWED_COPIES	AVAILABILITY_PERCENTAGE	UTILIZATION_PERCENTAGE
116	87	29	75.00	25.00

```
-- Most Borrowed Books KPI
-- Which books have the highest borrowing demand?
SELECT
    B.BOOK_ID,
    B.BOOK_TITLE,
    C.CATEGORY_NAME,
    COUNT(BI.ISSUE_ID) AS TOTAL_ISSUES,
    SUM(
        CASE
            WHEN BI.ISSUE_STATUS = 'Issued' THEN 1
```

```
            ELSE 0
        END
    ) AS CURRENTLY_ISSUED,
    SUM(
        CASE
            WHEN BI.ISSUE_STATUS = 'Returned' THEN 1
            ELSE 0
        END
    ) AS TOTAL_RETURNED
FROM BOOK B
INNER JOIN CATEGORY C
    ON B.CATEGORY_ID = C.CATEGORY_ID
LEFT JOIN BOOK_ISSUE BI
    ON B.BOOK_ID = BI.BOOK_ID
GROUP BY
    B.BOOK_ID,
    B.BOOK_TITLE,
    C.CATEGORY_NAME
ORDER BY TOTAL_ISSUES DESC
LIMIT 5;
```

BOOK_ID	BOOK_TITLE	CATEGORY_NAME	TOTAL_ISSUES	CURRENTLY_ISSUED	TOTAL_RETURNED
B007	The Guide	Literature	3	3	0
B001	1984	Fiction	1	0	1
B002	Harry Potter and the Philosophers Stone	Fiction	1	0	1
B003	A Brief History of Time	Science	1	0	1
B005	Wings of Fire	Biography	1	0	1

```
-- Member Engagement KPI
-- Which members borrow the most books, and how many books are currently
with them?
SELECT
    M.MEMBER_ID,
    CONCAT(M.FIRST_NAME, ' ', M.LAST_NAME) AS MEMBER_NAME,
    M.CITY,
    MS.ACCOUNT_TYPE,
    COUNT(BI.ISSUE_ID) AS TOTAL_BORROWING_TRANSACTIONS,
    SUM(
        CASE
            WHEN BI.ISSUE_STATUS = 'Issued' THEN 1
            ELSE 0
        END
    ) AS CURRENTLY_BORROWED
FROM MEMBERS M
INNER JOIN MEMBER_STATUS MS
    ON M.ACTIVE_STATUS_ID = MS.ACTIVE_STATUS_ID
LEFT JOIN BOOK_ISSUE BI
    ON M.MEMBER_ID = BI.MEMBER_ID
```

```
GROUP BY
    M.MEMBER_ID,
    M.FIRST_NAME,
    M.LAST_NAME,
    M.CITY,
    MS.ACCOUNT_TYPE
ORDER BY TOTAL_BORROWING_TRANSACTIONS DESC;
```

MEMBER_ID	MEMBER_NAME	CITY	ACCOUNT_TYPE	TOTAL_BORROWING_TRANSACTIONS	CURRENTLY_BORROWED
M001	Rahul Sharma	New York	Student	4	2
M002	Priya Patel	Mumbai	Student	3	2
M003	Amit Joshi	Thane	Faculty	2	0
M004	Sneha Desai	Mumbai	General	1	0
M007	Rajesh Kulkarni	Pune	Faculty	1	0
M008	Anjali Gupta	Mumbai	Student	1	1
M005	Vikas Mehta	Navi Mumbai	Student	0	0
M006	Neha Shah	Mumbai	General	0	0

```
-- Fine Collection and Outstanding Balance KPI
-- How much has the library charged in fines,
-- collected in payments, and still has outstanding?
SELECT
    COALESCE(FD.TOTAL_FINE_DUE, 0) AS TOTAL_FINE_CHARGED,
    COALESCE(FP.TOTAL_FINE_PAID, 0) AS TOTAL_FINE_COLLECTED,
    COALESCE(FD.TOTAL_FINE_DUE, 0)
        - COALESCE(FP.TOTAL_FINE_PAID, 0)
        AS OUTSTANDING_FINE,
    ROUND(
        COALESCE(FP.TOTAL_FINE_PAID, 0) * 100.0
        / NULLIF(FD.TOTAL_FINE_DUE, 0),
        2
    ) AS COLLECTION_PERCENTAGE
FROM
    (
        SELECT SUM(FINE_TOTAL) AS TOTAL_FINE_DUE
        FROM FINE_DUE
    ) FD
CROSS JOIN
    (
        SELECT SUM(PAYMENT_AMOUNT) AS TOTAL_FINE_PAID
        FROM FINE_PAYMENT
    ) FP;
```

TOTAL_FINE_CHARGED	TOTAL_FINE_COLLECTED	OUTSTANDING_FINE	COLLECTION_PERCENTAGE
275.00	350.00	-75.00	127.27

```sql
-- Library Staff Workload KPI
-- How many book-issue transactions has each staff member handled?
SELECT
    LS.ISSUED_BY_ID,
    LS.STAFF_NAME,
    LS.STAFF_DESIGNATION,
    COUNT(BI.ISSUE_ID) AS TOTAL_ISSUES_HANDLED,
    SUM(
        CASE
            WHEN BI.ISSUE_STATUS = 'Issued' THEN 1
            ELSE 0
        END
    ) AS CURRENTLY_ISSUED_COUNT,
    SUM(
        CASE
            WHEN BI.ISSUE_STATUS = 'Returned' THEN 1
            ELSE 0
        END
    ) AS RETURNED_COUNT
FROM LIBRARY_STAFF LS
LEFT JOIN BOOK_ISSUE BI
    ON LS.ISSUED_BY_ID = BI.ISSUED_BY_ID
GROUP BY
    LS.ISSUED_BY_ID,
    LS.STAFF_NAME,
    LS.STAFF_DESIGNATION
ORDER BY TOTAL_ISSUES_HANDLED DESC;
```

ISSUED_BY_ID	STAFF_NAME	STAFF_DESIGNATION	TOTAL_ISSUES_HANDLED	CURRENTLY_ISSUED_COUNT	RETURNED_COUNT
ST01	Rajesh Kumar	Chief Librarian	3	1	2
ST02	Anita Sharma	Librarian	3	1	2
ST03	Vijay Patil	Assistant Librarian	2	0	2
ST04	Meena Joshi	Library Assistant	1	0	1